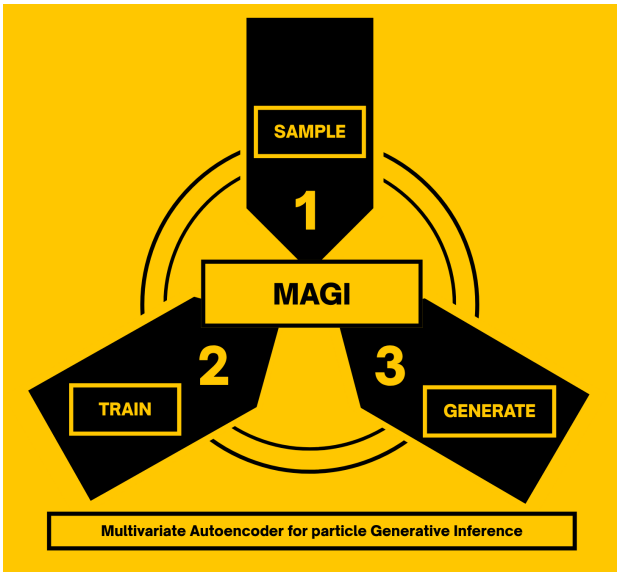


MAGI

Multivariate Autoencoder for particle Generative Inference

A cVAE-based generative particle source for Monte Carlo transport acceleration

User Manual — v1.0.0

Francesco Monastra
INAF-IAPS — ASI-SSDC
francesco.monastra@inaf.it

November 10, 2026

Scope of release v1.0.0

MAGI v1.0.0 reproduces the *joint* phase-space distribution (energy–position–direction correlations) and the energy continuum to within its stated acceptance bars, on three seeds and two reference sources; that record was measured on v0.8.2 and v1.0.0 improves on it with the prior refit and the 511 keV warp fix (Section 7). The release adds primary-conditioned models with grouped generation (Section 5) and first-entry crossings for surfaces that cut passive volumes (Section 6). Spectral-line *intensities* are still **not** reproduced: the per-line recovery measured on v0.8.2 runs from 0.7× to 4.9× the true value, is unstable across seeds and has not been remeasured. Line *positions* are reliable to ≤ 0.5 eV.
The numbers of this manual belong to two versions. The measured state of v1.0.0 is the classifier AUC of 0.62–0.63 on CR, the 511 keV line of CR at 3.40 % of the generated γ against 3.45 % real, the per-bin RMS of Section 11.2 and the detector closures of Section 11.3. The line intensities, the memorisation test and the coupling experiment are the record of v0.8.2 and are marked as such. Three results are not closed: the Th and Ra closures disagree in sign between IAPS (0.88 and 0.86) and SRON (1.40 ± 0.22 and 1.31 ± 0.19), the muon trigger

rate of DM1.2 lies 2.4σ above the full simulation (9.14 against 8.33 ± 0.34 per 10^6 primaries), and the Athena mass model is not covered.
Read Section 11 before quoting any number derived from a MAGI-generated source, and do not use this release to estimate a fluorescence or annihilation line flux.

Contents

1 Introduction **5**
1.1 What MAGI is . . . 5
1.2 What it is for . . . 5
1.3 What it is not for . . . 5
1.4 How to read this manual . . . 5
1.5 Terms used in this manual . . . 6

2 Theoretical foundation **6**
2.1 Conditional Variational Autoencoders . . . 6
2.2 Normalizing flows for the energy continuum . . . 7
2.3 The gated line–continuum mixture . . . 7
2.4 The learned conditional prior . . . 7
2.5 Atomic transition energies . . . 8
2.6 Optimization . . . 8
2.7 Relation to other work . . . 8

3 Structure of the code **8**
3.1 Repository layout . . . 9
3.2 Package layout . . . 9
3.3 The data flow . . . 10
3.4 Why per-variable heads . . . 10
3.5 Model variants . . . 10

4 Usage: functions and settings **11**
4.1 Installation . . . 11
4.2 Environment setup . . . 11
4.3 Loading data . . . 12
4.4 Preprocessing . . . 12
4.5 Spectral lines . . . 13
4.6 Building the dataset . . . 14
4.7 Training . . . 14
4.8 Checkpointing . . . 16
4.9 Generation . . . 16
4.10 Inspecting a trained model . . . 17

5 Primary-conditioned models and grouped generation **20**
5.1 The v1.0.0 recipe . . . 21

6 Tracking surfaces that cut other volumes: first-entry crossings **22**

7 Strategies to improve a trained model **24**
7.1 Exclude the pinned lines from the CDF warp . . . 24
7.2 Boost ranges and the line criterion . . . 24
7.3 Fit the quantile transform on all rows . . . 25
7.4 Refit the prior on the frozen encoder . . . 25
7.5 Weighted refit . . . 26
7.6 Latent bank . . . 26
7.7 Line sampling and the inward condition . . . 26

8 The tools **26**
8.1 Training and scoring . . . 27
8.2 Audits . . . 27
8.3 Synthetic stress tests . . . 28
8.4 Unit tests . . . 28

9 Example: a full run **29**
9.1 The short path . . . 29
9.2 The long path, step by step . . . 29
9.3 Producing the Geant4 source file . . . 30

10 Example: a validation **32**
10.1 The short path . . . 32
10.2 What it checks, and against what . . . 32
10.3 Doing it by hand . . . 33
10.4 Reading the output . . . 33

11 Accuracy: what has actually been measured **33**
11.1 Classifier two-sample test . . . 34
11.2 Per-bin deviation of the marginals . . . 35
11.3 Detector-level closure . . . 35
11.4 History: the measurements of v0.8.2 . . . 36
11.4.1 Detector-level validation: DM1.2 and SRON (v0.8.2) . . . 36
11.4.2 Memorisation test: does MAGI generate, or recall? . . . 38
11.4.3 Crossing-level: validated . . . 38
11.4.4 Crossing-level: not validated . . . 39
11.4.5 Choosing between the heads . . . 39
11.4.6 Closed: energy–geometry coupling does not explain the conversion deficit . . . 39
11.4.7 Negative results already established . . . 40

12 Limitations and traps **40**
12.1 Measured limits of v1.0.0 . . . 40
12.2 Traps . . . 41
12.3 Adapting MAGI to another detector or source . . . 42
12.4 Resuming an interrupted or early-stopped run . . . 42

13 Where to look next **44**

14 Citing MAGI, and its license **44**
14.1 License . . . 44
14.2 Citation . . . 44

1 Introduction

1.1 What MAGI is

MAGI — *Multivariate Autoencoder for particle Generative Inference* — is a conditional Variational Autoencoder (cVAE) [1, 2], implemented in Keras/TensorFlow, that learns the joint phase-space distribution of particles crossing a surface in a Monte Carlo simulation — their energy, position, direction and particle type — and then samples new, physically consistent events at a small fraction of the cost of running the full transport again.

Development and validation have been done against Geant4 [3, 4], and the shipped exporter writes Geant4-ready files, but nothing in the model is specific to it: MAGI learns from a table of crossing records and writes back a table in the same format it was given, so any consumer that can read that format can be driven from it.

The package is installed and imported as `magi`.

1.2 What it is for

The intended use is as a **drop-in replacement for a particle source** in a multi-stage simulation. Run the expensive full-physics simulation once to obtain a reference sample of particles crossing some surface; train MAGI on that sample; then use MAGI's output as the source for downstream simulations that would otherwise require repeating the expensive upstream physics.

This pays off when

- large numbers of particles are needed downstream;
- the events reaching the surface of interest are rare relative to the number of primaries thrown, so direct simulation is statistics-starved — whether the rarity is physical (a small interaction cross-section, a steep spectrum) or geometrical (a small solid angle, a deeply shielded volume);
- the same downstream simulation is run many times with different geometry or configuration choices, and re-deriving the upstream flux each time is wasteful.

Because MAGI's output schema matches its input schema, the natural way to use it is to cut a long simulation at a surface and resume from there — a decomposition that is useful in any transport problem with the statistics bottleneck above, not only the instrument-background case it was built for.

The scoring surface is currently a sphere. Section 12.3 states what the adaptation to another surface would require.

1.3 What it is not for

- Estimating the intensity of an individual spectral line (Section 11).
- Crossing surfaces that are not spheres. The surface MAGI is trained on is parametrized as a sphere (`center`/`radius`); see Section 12. This constrains the *scoring* surface only, since the surface is an intermediate bookkeeping step and a dummy spherical tracking volume can be placed in an otherwise arbitrary mass model to give MAGI something to learn on. A non-spherical crossing surface is not supported.
- Extrapolating outside the energy range or particle mix of the training sample. MAGI is a density model of the data it was shown, not a physics engine.

1.4 How to read this manual

The sections that a reader needs depend on the use. Sections 3 and 4 describe the code and the API reference (functions, settings and the contracts between them), Section 8 the command-line tools around a run, and Sections 9 and 10 are worked examples of a full run and of a validation. Section 11 states what has actually been measured, and Section 12 lists the traps.

Use	Sections
Generate a source for one detector from a Geant4 table	4, 9, 10, 11
Improve a trained model	7
Feed the HENABICS chain, with the primary carried on every crossing	5, and 6 when the surface cuts material
Quote a number from a generated source	11, 12
Inspect a trained network	4.10
Adapt the code to another surface	12.3

A short path for a first run is `Example_Usage.ipynb` at the repository root, a runnable and commented walkthrough of the whole pipeline on synthetic data that finishes in a few minutes. The four reference trainers of the release are `tools/train_cr_v1.py`, `tools/train_iaps.py`, `tools/train_dm12_primtruth.py` and `tools/train_athena_primtruth.py`.

Conventions

Energies are in MeV and positions in mm throughout, matching our Geant4's output convention. Code that must run on the CPU is shown with the `CUDA_VISIBLE_DEVICES=-1` prefix, which must be set *before* `magi` is imported.

1.5 Terms used in this manual

Crossing. A particle recorded where it crosses the scoring surface, with energy, position and direction.

Closure and R. The generated crossings are injected into the downstream Geant4 run, and R is the ratio of the detector rate obtained with them to the rate obtained with the real crossings. $R = 1$ is agreement, and *5x* means that five times the statistics of the real sample was injected.

Quantile transform. A monotone map of one variable to a normal distribution through its empirical quantiles, fitted per geometry variable and stored with the checkpoint (Section 7.3).

CDF warp. A monotone remapping of the energy distribution in which the continuum density is fitted; its knots must not sit on a line energy (Section 7.1). *warp2* is the variant of the Th model.

Zone table, atom, bank. The zone table gives the probability of each gate zone by crossing type, primary type and primary energy; the atom is the uncollided primary at E_{prim}; the bank is the set of latent codes that is reused at generation (Sections 5.1 and 7.6).

u_r, u_v, ϕ_r, ϕ_v**.** The radial and angular geometry variables of a crossing on the sphere, as defined in Section 3; `_qtfull` marks a run whose geometry quantile transforms were refitted on all training rows.

2 Theoretical foundation

This section states what MAGI is built on and where each piece is implemented.

2.1 Conditional Variational Autoencoders

A variational autoencoder [1] models a density $p(x)$ through a latent variable z, training an encoder $q(z|x)$ and decoder $p(x|z)$ against the evidence lower bound

$$\mathcal{L} \;=\; \mathbb{E}_{q(z|x)}[\log p(x \mid z)] \;-\; \mathrm{KL}(q(z \mid x) \,\|\, p(z))\,. \tag{1}$$

The conditional variant [2] conditions every term on an observed covariate c, giving $q(z|x,c)$, $p(x|z,c)$ and $p(z|c)$. In MAGI c is the particle-type one-hot, which is what lets generation be steered to a chosen particle mix rather than only reproducing the training proportions.

Implemented in `magi/core/model.py`; the reconstruction and KL terms in `magi/core/losses.py`.

2.2 Normalizing flows for the energy continuum

A normalizing flow [5, 6] represents a density as an invertible, differentiable map f applied to a simple base density, with the change-of-variables formula supplying an exact likelihood:

$$\log p(y) \;=\; \log p_u(f(y)) \;+\; \log \left| \det \frac{\partial f}{\partial y} \right| . \tag{2}$$

MAGI uses monotonic rational-quadratic spline transforms [7] for the energy continuum. The choice matters here for a specific reason: the crossing spectrum of a cosmic-ray source (our training scenario) spans roughly nine decades and contains sharp non-Gaussian structure — a Compton edge, a muon peak, a steep low-energy tail — that a single Gaussian cannot represent at all, and that an affine flow represents only poorly. A spline flow is flexible enough to follow that structure while remaining exactly invertible, which the exact-likelihood training requires.

Implemented in `magi/core/flows.py` (`ConditionalRQSFlow`), selected with `continuum_mode="flow"`.

2.3 The gated line–continuum mixture

The central modelling decision in v0.8 is that spectral line positions are *not learned*. A categorical energy head can never place a line more precisely than one bin, and on a log grid spanning nine decades a bin is hundreds of eV — orders of magnitude wider than a real fluorescence line. Positions therefore become fixed physical inputs, measured from the data and taken from evaluated atomic data, and the head becomes a mixture:

$$p(y \mid z,c) \;=\; g_0(z,c)\, p_{\text{cont}}(y \mid z,c) \;+\; \sum_{\ell=1}^{L} g_\ell(z,c)\, \mathcal{N}\!\left(y;\, y_\ell,\, \sigma_\ell^2\right), \qquad \sum_{\ell=0}^{L} g_\ell = 1, \tag{3}$$

with $y = \log_{10}(E/\text{MeV})$. The widths σ_ℓ are pinned to the instrument's energy resolution rather than fitted.

The gate g faces a severe class imbalance: a line can be 10^{-5} of the sample, so an unweighted cross-entropy is minimized well by never routing anything to it. MAGI therefore applies focal down-weighting of easy examples [8], exposed as `gate_focal_gamma`.

An empirical caution on γ

Use $\gamma = 1$. At $\gamma \geq 2$ the gate over-corrects and begins routing genuine continuum events into the line components, visibly digging a hole in the continuum on either side of each line. This is measured, not assumed: the near-line continuum ratio degrades monotonically across a γ sweep.

2.4 The learned conditional prior

With a standard $p(z) = \mathcal{N}(0, I)$, two problems arise. First, the decoder alone must carry every correlation between energy and geometry. Second, and more subtly, there is a **train/generate mismatch**. During training the decoder only ever sees latent codes produced by the encoder from a real event, $z \sim q(z \mid x)$. At generation time there is no real event to encode, so codes are drawn from the prior instead, $z \sim p(z)$. These two distributions are the same only if the *aggregated* posterior $q(z) = \mathbb{E}_x\, q(z \mid x)$ — the pooled cloud of codes over the whole dataset — happens to coincide with the prior. When it does not, the generator samples regions of latent space the decoder was never trained on and produces events from an extrapolation [9].

MAGI replaces the fixed prior with a learned conditional coupling flow $p(z \mid c)$, built from affine coupling layers [10] and trained with a Monte-Carlo estimate of the KL term. This is what brings the coupling residuals of Section 11 inside their bar, and it is the difference the v0.7.2 categorical head cannot make up.

In v0.8.2 the prior's conditioning is widened from the particle-type one-hot alone to $[\,c,\ \text{zone}\,]$, where zone is the continuum/line routing target (`prior_zone_conditioning=True`). Without the zone in the conditioning, the prior has no way to express which mixture component an event belongs to, so at generation time rare lines are routed by whatever the type marginal happens to imply.

Implemented in `magi/core/priors.py` (`ConditionalCouplingPrior`).

2.5 Atomic transition energies

Line positions are only as good as the table they are pinned at. MAGI takes transition *labels* (Kα_1, Kβ, and so on) from Bearden's compilation [11], but transition *energies* from the EADL evaluated library [12], because EADL is what the Geant4 `SpacePhysicsList` actually emitted.

Every pinned position is audited against a centroid measured from the real spectrum (`tools/line_centroid_audit.py`).

2.6 Optimization

Training uses Adam [13]. The per-variable loss terms are combined with fixed weights (`w_energy`, `w_ur`, ..., `w_gate_aux`) chosen so that no single head dominates the objective purely because its natural scale is larger.

A task-adaptive loss-weight scheduler that rebalances those weights *during* training exists in `magi/training/adaptive_callbacks.py` and gives the `..._TaskAdaptive` classes their name, but it is opt-in and was not used for any v0.8 result reported here — see the warning in Section 4.

2.7 Relation to other work

MAGI's problem — learn a compact, resampleable particle source on a surface so that a multi-stage Monte Carlo can be decoupled — is the same one KDSource [14] addresses with kernel density estimation. MAGI substitutes a deep conditional generative model, which scales better to the joint energy–position–direction–type density and conditions naturally on particle type.

The broader use of deep generative models to replace expensive detector simulation is well established in high-energy physics [15], with GAN- [16] and flow-based [17] calorimeter shower models now in production use [18]. MAGI differs in target: rather than showers inside a calorimeter, it models the particle flux crossing a surface, so that a transport problem can be cut at that surface and resumed from a cheap sampler.

That target is not tied to one instrument or one energy regime. The conditions MAGI is built for are structural: the events that matter are rare at the surface of interest — because the cross-section is small, the spectrum is steep, the solid angle is small, or the volume is deeply shielded — and the problem admits a multi-stage decomposition, so the expensive upstream physics can be paid for once and resampled many times. Any simulation with that shape is a candidate. The development and validation reported here were carried out on the cryogenic X-ray instrument background problem [19], which supplies this manual's two reference sources, but the numbers in Section 11 are properties of those sources rather than limits of the method — measure on your own source with the acceptance harness rather than assuming they transfer.

3 Structure of the code

3.1 Repository layout

The repository has two parts: the installable library, and the experiment material that uses it.

Path	Purpose
`MAGI_package/`	The installable Python library (`import magi`). Everything in Section 4 lives here.
`Example_Usage.ipynb`	Runnable walkthrough on synthetic data. Start here.
`MAGI_v0_*.ipynb`	The experiment notebooks. Highest version number is the current pipeline.
`tools/`	Command-line scripts *around* a run rather than inside one: full training runs, the acceptance harness, audits, synthetic stress tests (Section 8).
`scripts/generate_geant_source.py`	Produces a Geant4-ready source file from a trained run, outside a notebook. This is what the companion Geant4 project's `/generator/mlScript` macro invokes (Section 9.3).
`CandidateLines/`	Candidate spectral-line tables built from a GDML mass model, in the JSON form `magi.load_candidate_energy_lines` reads.
`trained_models/`, `checkpoints/`	Trained-run artifacts. Weights are gitignored; the JSON config/metadata is versioned.
`docs/`	Development notes and this manual under `docs/manual/`.

3.2 Package layout

```
magi/
  __init__.py       public API surface (the canonical list of exports)
  magi.py           high-level convenience API: setup, build_model, train_model
  config.py         environment/seed initialization
  core/
    model.py        the CVAE classes, one per version in the lineage
    losses.py       shared NLL / angular / mixture losses
    geometry.py     physical coordinate transforms
    flows.py        conditional rational-quadratic-spline flow (v0.8 continuum)
    priors.py       conditional coupling prior p(z|cond)   (v0.8)
  data/
    io.py           load/save raw detector tables, normalization, line tables
    preprocessing.py physical features, energy binning, line detection,
                     quantile transforms, gate targets
    dataset.py      split, scale, one-hot conditioning, tf.data construction
  training/
    train.py            compile/fit wrappers
    adaptive_callbacks.py task-adaptive loss rebalancing and monitors
    checkpointing.py    save/load weights + metadata + transformers as a unit
  generation/
    sampling.py       draw latent codes and conditioning
    reconstruction.py invert model outputs back to physical quantities
    export.py         write Geant4 source files (text or chunked binary)
  validation/
    metrics.py      Wasserstein, line-integral recovery
    compare.py      histogram-residual comparisons, range/norm checks
  utils/
    plotting.py         all plot_* helpers, light/dark theme
    model_inspection.py introspect a built model
    circuit_viz.py       interactive routing-circuit HTML (Section~\ref{sec:visualization})
    full_circuit.py      interactive per-event gradient-circuit HTML (Section~\ref{sec:
    visualization})
tests/              238 tests; see Section~\ref{sec:tools}
```

```
docs/USAGE.md       short-form user guide
```

3.3 The data flow

Every run follows the same seven stages. Each arrow is a function whose output is the next one's input; each stage has a matching `report_*` or `print_*` helper that prints sanity checks and returns nothing.

1. **Load** — `load_detector_table` reads the raw crossing file into a DataFrame.
2. **Physical features** — `build_physical_features` turns Cartesian position/direction into the sphere-surface parametrization $(u_r, \phi_r, u_v, \phi_v)$ and measures the primary fraction.
3. **Feature frame** — `build_feature_dataframe` bins energy, detects spectral lines and fits the quantile geometry transforms.
4. **Dataset** — a five-step chain, each feeding the next:

   `filter_particle_types_continuous_geometry` → `split_feature_data` → `scale_continuous_features` → `build_conditioning_and_weights` → `build_tf_datasets`.

5. **Train** — a class from `magi.core`, then `compile_model` and `fit_model`.
6. **Checkpoint** — `save_final_trained_model` writes weights, config, metadata, history, task weights and a summary as one unit.
7. **Generate and validate** — `generate_latent_outputs` → `reconstruct_generated_features` → `reconstruct_generated_physics` → export, with `magi.validation` scoring the result.

3.4 Why per-variable heads

MAGI does not predict one flat output vector. Each physical variable gets its own head, because each has a different pathology:

- u_v is bounded on $[-1, 1]$ and piles up at the edges. A Gaussian head fits it badly, so the model works in $s = \operatorname{atanh}(u)$ space, or on a quantile transform of it.
- ϕ_r, ϕ_v are periodic. The v0.7 heads predict a $(\cos, \sin)$ pair softly regularized onto the unit circle; v0.7.2 predicts the angle through a quantile transform directly.
- Energy spans many decades and contains narrow spectral lines. This is the axis the version lineage is really about (Section 3.5).

3.5 Model variants

The classes in `magi.core` form a lineage. A trained run records which one it is in `model_config["model_class"]`; check that before assuming behaviour.

Class	Version	Energy / geometry
`CVAE_CatEnergy_CatUV`	v0.6	Categorical energy, discretized u_v.
`CVAE_CatEnergy_CatUV_TaskAdaptive`	v0.6	As above, plus task-adaptive loss weighting.
`CVAE_CatEnergy_ContGeom_TaskAdaptive`	v0.7	Continuous geometry, $(\cos, \sin)$ angles.
`CVAE_CatEnergy_ContPhi_TaskAdaptive`	v0.7.2	Continuous ϕ_r, ϕ_v. The stable fallback head.
`CVAE_MixEnergy_ContPhi_TaskAdaptive`	v0.8 (current head in v1.0.0)	v0.7.2 geometry; energy becomes a gated mixture of a continuum density and fixed-position line components.

The two changes that define v0.8 are described in the theory section rather than repeated here: the **gated line–continuum mixture** that replaces the categorical energy head (§2, "The gated

line–continuum mixture"), and the **learned conditional prior** that replaces $\mathcal{N}(0, I)$ (§2, "The learned conditional prior"). Both are selected through constructor arguments — `continuum_mode`, `prior` — documented with the rest of the training settings in Section 4.

In v1.0.0 the first four classes are kept for reference and for loading old runs, and the mixture head is the one that the reference trainers build. Its options for primary-conditioned models (the uncollided atom, the zone table by primary, the energy loss flow and, at generation, the latent bank) are described in Section 5, and the measures that improve a trained model in Section 7.

Use `magi.print_model_structure(model)` to print a built model's generative structure, formulas, configured line table and parameter counts.

4 Usage: functions and settings

Every exported function carries a full NumPy-style docstring, so hovering over a name in an editor shows its parameters and return value. This section covers the contracts *between* functions, which a per-function docstring cannot.

VSCode hover

The repository now ships `.vscode/settings.json` pointing the analyzer at the source tree directly, which fixes it regardless of how the package was installed:

```
{"python.analysis.extraPaths": ["MAGI_package"]}
```

If you work from a different editor or workspace root, set the equivalent option to `MAGI_package/`. Reloading the window is required for the analyzer to pick it up.

4.1 Installation

```
pip install -e MAGI_package/      # develop the library
pip install -r requirements.txt   # or install it as a dependency
```

Core dependencies: `tensorflow` (plus `tensorflow-metal` on Apple Silicon), `tensorflow-probability`, `scikit-learn`, `joblib`, `astropy`, `optuna`.

4.2 Environment setup

```
import magi
magi.initialize_environment(seed=42, cpu_only=True, quiet=True)
magi.print_tf_info()
```

cpu_only limitation on tf-metal

`cpu_only=True` only takes effect if it runs before TensorFlow initializes its devices. In a script that imports `magi` at module level this is already too late — set the environment variable instead, before any import:

```
import os
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"
import magi
```

Every script in `tools/` does exactly this, and for a reason: on this workload `tensorflow-metal` is slower than the CPU, and importing `tensorflow_probability` initializes the Metal device, after which `set_visible_devices` silently stops working. This is only on our testing machine (M1 Apple Silicon Macbook), therefore feel free to adapt to your case.

4.3 Loading data

```
df = magi.load_detector_table("TrainingData/alloutputDSCryoSphereCR.dat")
magi.report_basic_table_checks(df)
```

Three column schemas are auto-detected from the field count on the first data line. Each is a strict prefix of the next, so the columns always appear in the same order and a wider file is readable by anything that understands a narrower one:

Cols	Schema	Column order, left to right
9	legacy	`EventId ParticleName Energy X Y Z Vx Vy Vz`
10	PrimBool-tagged	`EventId ParticleName Energy PrimBool X Y Z Vx Vy Vz`
13	lineage	`EventId ParticleName Energy PrimBool ParticleId ParentParticleId CreatorProcessName X Y Z Vx Vy Vz`

Column	Meaning and units
`EventId`	Geant4 event number the crossing belongs to.
`ParticleName`	Geant4 particle name (`gamma`, `e-`, `mu-`, ...). Becomes the conditioning one-hot.
`Energy`	Kinetic energy at the crossing, **MeV**.
`X Y Z`	Crossing position, **mm**, in the mass model's frame.
`Vx Vy Vz`	Direction, a normalized Cartesian unit vector.
`PrimBool`	1 if the particle is a Geant4 primary (`ParentID == 0`). Used for the cosmic-ray primary fraction.
`ParticleId`	Track ID of this particle.
`ParentParticleId`	Track ID of its parent, 0 for a primary.
`CreatorProcessName`	Process that created it (`RadioactiveDecay`, `compt`, ...). This is the column that makes the radioactive primary fraction computable.

Choose the widest schema your simulation can write. The 13-column lineage form is **required** for radioactive sources — see the `source_type` paragraph below for why `PrimBool` alone cannot answer the question there.

4.4 Preprocessing

```
prep = magi.build_physical_features(
    df, center=(0.0, 0.0, 0.0), radius=100.0,
    source_type="cosmic_ray",
)
magi.print_physical_summary(prep)

feature_pack = magi.build_feature_dataframe(
    prep,
    energy_binning_mode="log_fixed_count",
    n_bins=512,
    min_counts=20,
    geometry_transform="quantile_u_r_u_v_phi_r_phi_v",
    energy_transform="log10",      # required by the v0.8 mixture head
)
magi.report_feature_dataframe(feature_pack)
```

`source_type` decides flux normalization. With `"cosmic_ray"` a crossing counts as primary if `PrimBool == 1`. With `"radioactive"` it counts as primary if `CreatorProcessName== "RadioactiveDecay"`, because for a source embedded in bulk material the literal Geant4 primary is the decaying nucleus, which never propagates — `PrimBool` is always 0. Choosing wrongly does not error; it silently yields a wrong normalization factor.

Why `log_fixed_count` bins, when lines are measured at fixed eV

These are two different passes with two different jobs, and they use different grids on purpose. The `log_fixed_count` grid is for **detection**. A cosmic-ray crossing spectrum spans about nine decades, so no single fixed width works: a width fine enough to resolve a 1.5 keV line puts essentially zero counts per bin at 100 GeV, and a width coarse enough to be populated up there is hundreds of keV wide down low. `log_fixed_count` instead adapts the width to hold roughly constant counts per bin across the whole range, which is what lets one prominence threshold work at both ends.

The fixed-resolution numbers are for **measurement**, in a second, local pass: `refine_bin_width_mev` re-histograms a narrow neighbourhood of each candidate at the instrument's own scale (4 eV here), and `confirm_unresolved_candidate_lines` does the same for candidates the coarse pass could not separate. `tools/line_centroid_audit.py` then verifies the outcome against the real spectrum.

Coarse adaptive grid to find peaks; fine fixed grid to place them. Using the detection grid for placement is the v0.7.2 error the mixture head exists to remove.

`geometry_transform` is a stringly-typed contract. The value (`"arctanh_uv_discrete"`, `"quantile_u_r_u_v"`, `"quantile_u_r_u_v_phi_r_phi_v"`) fixes the column ordering shared by preprocessing, the model's input layout and reconstruction. It is persisted in the saved metadata and **must** match between the run that trained a model and the run that generates from it. A mismatch does not raise — it produces physically wrong output.

4.5 Spectral lines

Line handling is the part of the pipeline most specific to v0.8, and the part where a silent mistake is most expensive.

Note the ordering: lines are detected on the *raw* energies, on their own fine binning, *before* `build_feature_dataframe` is called. The feature frame's binning is a separate, coarser thing — with the mixture head it no longer constrains where a line can be placed, only how the continuum is described. The resulting gate targets are then carried into the dataset as extra feature columns.

```
payload = magi.load_candidate_energy_lines("CandidateLines/cryosphere_lines.json")
candidate_lines = payload["lines"]

res = magi.detect_energy_lines(
    E, binning_mode="log_fixed_count", n_bins=1024,
    candidate_lines=candidate_lines,
    refine_bin_width_mev=4.0e-6)   # refine to the pinned line width
magi.print_detected_energy_lines(res)

matched = [m for m in res["matched_lines"] if m["count"] >= 100]
matched += magi.confirm_unresolved_candidate_lines(
    E, candidate_lines, matched, resolution_ev=4.0)

gate_targets = magi.build_gate_targets(
    E, feature_pack["energy_bins"], matched,
    bandwidth_mode="resolution",       # the default; see the warning below
    bandwidth_fwhm_mev=4.0e-6)
line_logsigma = magi.line_logsigma_from_resolution(
    [m["candidate_energy_mev"] for m in matched], resolution_ev=4.0, fwhm=True)
```

`bandwidth_mode` still under validation

`bandwidth_mode` controls how `build_gate_targets` decides which *real* events count as line events when building the gate's training target. It does not affect the generated line width,

which is pinned separately via `line_logsigma_from_resolution`.
Raw Geant4 crossing data has **no detector response applied**, so simulated fluorescence lines are exactly monoenergetic: in the reference cosmic-ray source all 7 542 Cu Kα_1 events share one identical `float64` energy, and every nearby continuum event is a singleton. On that reasoning, using the instrument's 4 eV FWHM as a Gaussian kernel to decide line membership is the wrong tool — it hands weight to continuum a few eV away and blurs doublets closer than a few resolution widths — and `bandwidth_mode="exact"` was added to match the simulation's own ground truth.
However, during testing, the results for line generation were worse. `"resolution"` is therefore the default and is what every confirmed result in Section 11 was produced with. `"exact"` remains available and tested, but it is a research option, not a recommendation.

`confirm_unresolved_candidate_lines` exists because a doublet closer together than one detection bin is always a single peak, so only one member is ever detected regardless of how good the matching logic is — the gap is in peak detection, not matching. It re-measures every unmatched candidate at eV resolution and confirms the ones that are real. Both `tools/run_v0_8_real.py` and `tools/acceptance_v0_8.py` must call it identically, or the checkpoint's `n_lines` silently misaligns against the rebuilt pipeline's line count.

4.6 Building the dataset

```
dataset_pack   = magi.filter_particle_types_continuous_geometry(
                     feature_pack["feat"], prob_threshold=1e-5)
split_pack     = magi.split_feature_data(dataset_pack, random_state=42)
scaled_pack    = magi.scale_continuous_features(split_pack, scale_cols=())
condition_pack = magi.build_conditioning_and_weights(
                     scaled_pack, dataset_pack["n_types"], alpha=0.5)
ds_pack        = magi.build_tf_datasets(condition_pack, batch_size=4096)
magi.report_tf_datasets(ds_pack)
```

Note `scale_cols=()` for v0.7+ quantile geometry: those columns are already standardized by their own transform, and scaling them twice is a mistake the empty tuple prevents. The split is stratified on particle type; `alpha` controls how strongly rare species are up-weighted (0 = natural proportions, 1 = fully balanced, 0.5 = default).

4.7 Training

```
model = magi.CVAE_MixEnergy_ContPhi_TaskAdaptive(
    n_types=dataset_pack["n_types"], latent_dim=8,
    line_positions_y=line_positions_y,
    line_logsigma_init=line_logsigma,
    continuum_mode="flow",
    continuum_flow_warp="cdf",
    energy_flow_condition="z_cond",
    prior="coupling",
    prior_zone_conditioning=True,
    zone_probs=zone_probs,
    gate_focal_gamma=1.0,
    w_gate_aux=2.0,
)

magi.compile_model(model, learning_rate=2e-4, optimizer="adam")
history = magi.fit_model(
    model, ds_pack["train_ds"], ds_pack["val_ds"],
    epochs=40, callbacks=magi.build_default_callbacks(), verbose=2)
```

The constructor takes `n_types` and `line_positions_y` positionally; there is no `n_cont` argument. The number of continuous targets is not configurable — it is fixed by the geometry parametrization at four (u_r, u_v, ϕ_r, ϕ_v) plus energy plus the $L + 1$ gate-target columns, and the model derives it from `line_positions_y`.

Setting	Default	Effect
`latent_dim`	8	Latent width. Larger widens the posterior/prior gap that the coupling prior exists to close.
`continuum_mode`	`"flow"`	A single Gaussian cannot represent a multi-decade continuum. `"gaussian"` is kept for regression only.
`continuum_flow_warp`	`"affine"`	**Set this to `"cdf"` yourself.** It pre-warps the flow's input by the empirical CDF, which is what recovers the Compton edge and the low-energy tail. It is the one recommended value that cannot be a default: it requires `continuum_flow_warp_y_knots/_z_knots`, which are fit from your training energies with `magi.fit_cdf_warp_knots`, and the constructor never sees the data. Passing `"cdf"` without knots raises.
`energy_flow_condition`	`"z_cond"`	Conditions the continuum flow on both latent and conditioning.
`prior`	`"coupling"`	The learned conditional prior (§2). This is what makes coupling residuals pass.
`prior_zone_conditioning`	`False`	Condition the prior on `[type, zone]` rather than type alone. Requires `zone_probs`.
`energy_condition_geometry`	`False`	Couple the energy head to sampled geometry instead of leaving the two conditionally independent given z. Under evaluation — see the note below before switching it on.
`gate_focal_gamma`	1.0	Focal weighting on the gate loss. Keep at 1; $\gamma \geq 2$ digs the continuum out from between the lines.
`w_gate_aux`	0.3	Weight of the auxiliary gate-routing loss. The reference runs use 2.0.

`energy_condition_geometry`: goal and current status

By default the decoder heads are **conditionally independent given the latent**: $p(x \mid z, t) = \prod_i p(x_i \mid z, t)$. Energy and geometry meet only through z, so a combination that is individually common in both — a 511 keV photon *and* a horizontal equatorial track — can fall through the gap between them even when each marginal is right.
Setting `energy_condition_geometry=True` factorises the decoder autoregressively instead,

$$p(\text{geom} \mid z)\; p(E \mid \text{geom}, z),$$

teacher-forced on true geometry during training and re-decoded from *sampled* geometry at generation, so both paths apply the same conditional. The geometry heads never see the observed geometry, so nothing leaks into their own reconstruction term. The ordering is one-way: geometry informs energy, not the reverse.

Status. Shipped, off by default, and *not* validated. It was introduced as the v0.8.3 feature and not adopted for lack of time. Treat `True` as experimental.
Turning it on changes the generative structure, so a checkpoint trained with it must be generated from with it — which the saved config handles automatically.

`build_default_callbacks` is not the task-adaptive scheduler

Despite the class name `..._TaskAdaptive`, `build_default_callbacks` returns only `EarlyStopping` (restoring best weights) and `ReduceLROnPlateau`. It deliberately does **not** include `TaskAdaptiveLossScheduler` from `magi/training/adaptive_callbacks.py`, and neither `tools/run_v0_8_real.py` nor any v0.8 result in this manual used it — the per-task loss weights stayed at the fixed values passed to the constructor (`w_energy`, `w_ur`, ..., `w_gate_aux`). The scheduler was used in the v0.7/v0.7.2 line and remains available; add it to the callback list yourself if you want it. Do not assume it is running because the class name says "TaskAdaptive".

4.8 Checkpointing

```
magi.save_final_trained_model(
    model=model,
    save_dir="trained_models/my_run",
    model_name="mix_CR",
    history=history,
    model_config=model.to_generation_config(),
    preprocessing_metadata=preprocessing_metadata,
)
joblib.dump(transformers, "trained_models/my_run/mix_CR_quantile_transformers.joblib")
```

A checkpoint is one unit

A run directory holds `*.weights.h5`, `*_config.json`, `*_metadata.json`, `*_history.json`, `*_task_weights.json`, `*_summary.txt` and — for quantile-geometry runs — `*_quantile_transformers.joblib`. Generation needs the config and metadata as much as the weights, because they carry the transforms that invert the preprocessing. Copy or move them together.

4.9 Generation

```
model = magi.load_task_adaptive_model_for_generation(
    save_dir="trained_models/my_run", model_name="mix_CR",
    model_config=magi.load_json("trained_models/my_run/mix_CR_config.json"),
    n_types=n_types, type_weights=type_weights, radius=100.0)

gen_pack = magi.generate_latent_outputs(
    model, n_samples=n_real, type_probs=type_probs,
    n_types=n_types, idx_to_type=idx_to_type)

reco = magi.reconstruct_generated_features(
    gen_pack, energy_head_mode="mixture",
    geometry_metadata=geometry_metadata, energy_metadata=energy_metadata)

gen = magi.reconstruct_generated_physics(
    reco, center=(0.0, 0.0, -507.66), radius=100.0)
```

Generation and validation data should be the same number

The line-recovery metrics compare raw counts. Generating fewer events than the real sample deflates every ratio by exactly the shortfall. Set `n_samples` to the real event count when validating.

For a Geant4-ready file, use the one-call entry point, which loads the checkpoint and streams events to disk in chunks:

```
magi.generate_detector_input_file(
    save_dir="trained_models/my_run", model_name="mix_CR",
    model_config=model_config, n_types=n_types,
    type_weights=type_weights, type_probs=type_probs,
    idx_to_type=idx_to_type, geometry_metadata=geometry_metadata,
    energy_head_mode="mixture", energy_metadata=energy_metadata,
    output_file="generated/source.dat", n_events=10_000_000,
    radius=100.0, center=(0.0, 0.0, 0.0),
    output_format="binary", seed=42)
```

Neutrinos are filtered out by default (they do not contribute to detector background here). With `generate_until_n_written=True`, the default, generation continues until `n_events` *transportable* particles have been written, so Geant4 receives exactly the requested count.

4.10 Inspecting a trained model

Two self-contained, interactive HTML tools in `magi.utils` inspect a trained mixture run (the v0.8 head and its successors) without a plotting library or a running kernel; each call writes one `.html` file that opens in any browser, with its own light/dark palette and no external dependency.

`save_routing_circuit` The gate's conditional routing, per type: for each particle type, a fan-out diagram of one-hot conditioning → gate → zone, colored by the checkpoint's own `zone_probs` — the same per-type empirical routing frequency the learned conditional prior is conditioned on (§2).

```
magi.save_routing_circuit(save_dir="trained_models/my_run",
                          model_name="mix_CR")
```

Reads only the checkpoint's own config/metadata — no raw data, no loaded model, so it runs in under a second. Requires `prior_zone_conditioning=True` at training time: a `config_version 1` checkpoint (or any run with zone-conditioning off) has no `zone_probs` to plot and raises `KeyError`.

`save_full_circuit` How heavily every unit in the network is actually *used*, per particle type, aggregated over many real held-out events (Figure 2).

The metric is **Expected Conductance**. Integrated Gradients [20] attributes a prediction to *input features*; conductance [21] is its generalization to *internal units*, formally path-integrated gradients evaluated at a hidden layer, at comparable cost. A single arbitrary baseline is avoided: each event is attributed against a pool of other real held-out events of the same type and averaged, following expected gradients [22]. A zero baseline would be meaningless here: inputs are quantile-transformed, so 0 is a real, specific quantile rather than the absence of a signal.

Two details make the quantity well defined. Only the continuous feature block is interpolated along the attribution path — the one-hot type conditioning is held fixed, since linearly interpolating a categorical is not meaningful — and the attribution target is the model's own per-event reconstruction loss, which reaches also the branches with no gradient path to the gate (the energy branch reads the decoder stem directly, bypassing the deep trunk that feeds position and direction) and gives them a genuine, nonzero attribution. The defaults are `n_events_per_type=128`, `n_baselines=16` and `n_steps=32`.

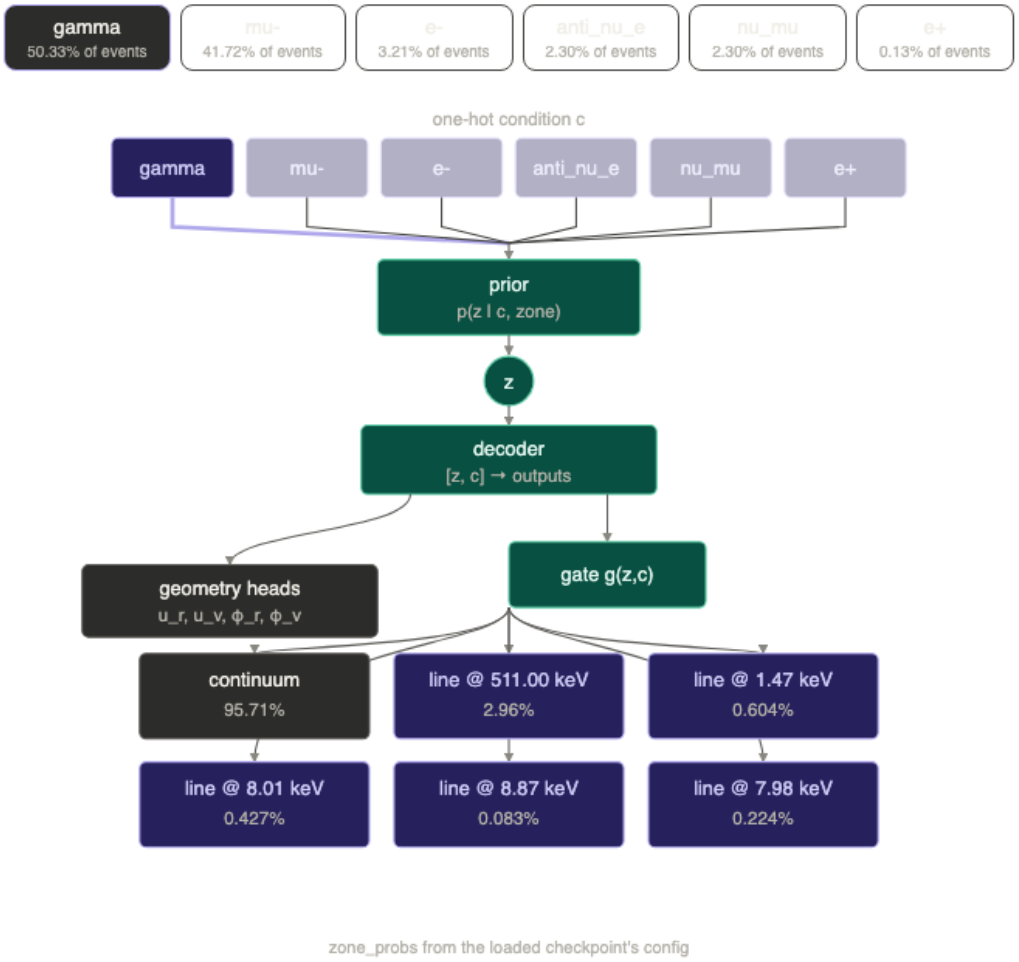


Figure 1: `save_routing_circuit` on the reference CR run. Selecting a particle type highlights its one-hot wire and fills in that type's routing percentage on every zone leaf. Here `gamma` sends 95.7 % of its events to the continuum and the rest to the five pinned lines; `mu-` routes essentially everything to the continuum, which is the physically correct answer and a useful sanity check that the gate has not learned to sprinkle lines across types that cannot produce them.

```
magi.save_full_circuit(
    save_dir="trained_models/my_run", model_name="mix_CR",
    training_data_filepath="TrainingData/alloutputDSCryoSphereCR.dat",
    candidate_lines_file="CandidateLines/CANDIDATE_ENERGY_LINES_"
                         "SRON_CCNwithXFDM_NoShield_FlowerCryoAC_fixed_EADL.json",
    center=(0.0, 0.0, -507.66), radius=100.0,
    n_events_per_type=128, n_baselines=16, n_steps=32)
```

Reading it

The colour of a node is a **percentile rank** inside its layer and not a magnitude, so that by construction some units of every layer always look dark; the colours therefore cannot answer whether a layer is oversized. For that question, open the **per-layer absolute usage** panel below the diagram (Table 1), which reports the unnormalized magnitudes. The column to read is `load`: a layer's share of the network's total conductance divided by its share of the network's units. 1.0× means the layer pulls exactly its weight; 0.2× means it holds five times more units than its contribution justifies.

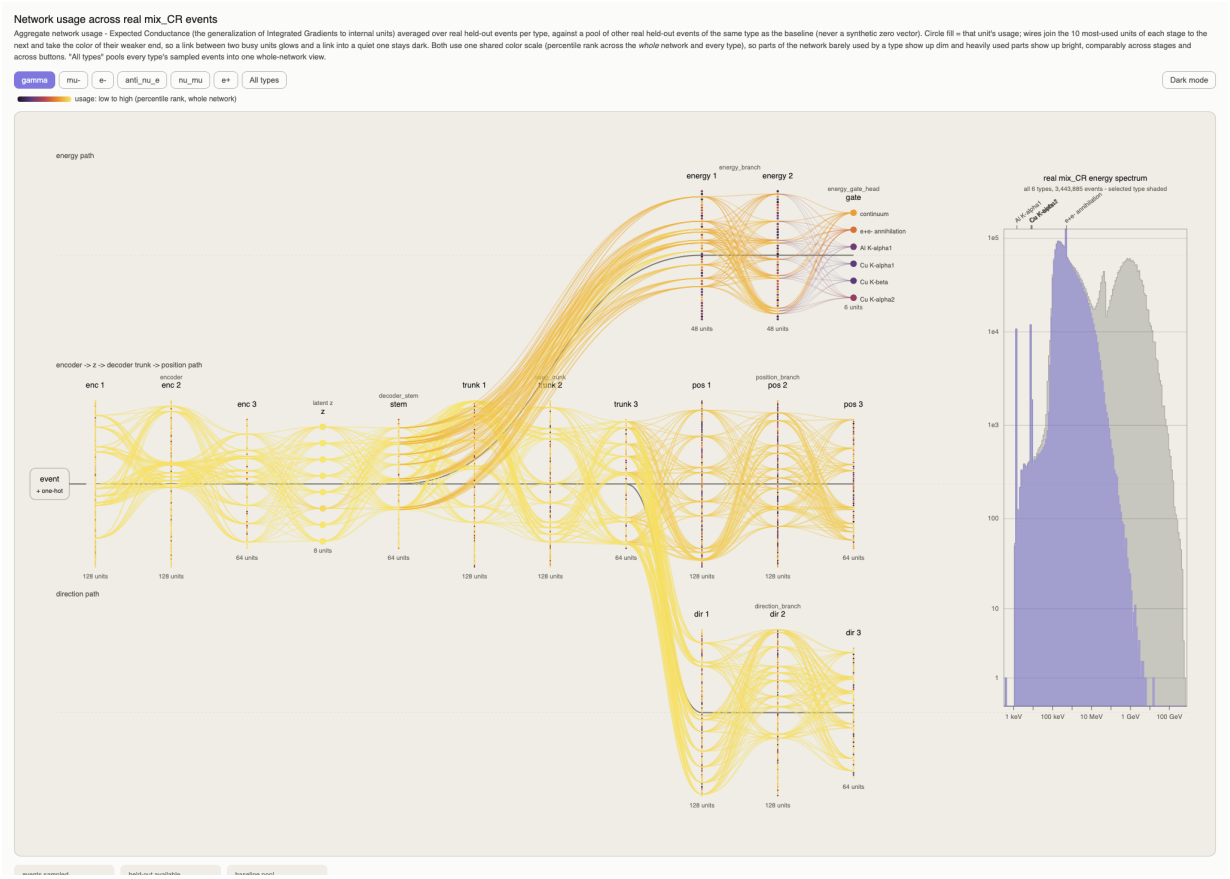


Figure 2: `save_full_circuit` on the reference CR run, with `gamma` selected (the view the file opens on). Each column is one layer, each dot one unit colored by usage; wires join the ten most-used units of each stage to the next and take the color of their weaker end. The encoder, latent z and decoder stem run bright, and the energy path fans up into the gate, whose six zones are labelled at the right against the real CR spectrum. Switching to "All types" pools every type's events into one view — that is the grouping Table 1 reports.

A flag is where to run an experiment, not a verdict

Attribution says the loss gradient does not flow much through a unit at the current optimum. It does **not** say the network computes the same function without it, and it is blind to redundancy in both directions: two duplicated units each look moderately used, while a unit that is dormant on average may be the one carrying rare line events. Concentration alone is not evidence either — a layer can be peaky and still carry its weight, which is why the flag is driven by `load`, with `top-20 %` as a supporting column.
The only thing that settles a width question is an ablation: shrink the layer, retrain one seed, and re-score with `tools/acceptance_v0_8.py`.

This one re-derives your preprocessing — budget the time

Unlike the routing circuit, `save_full_circuit` needs real held-out events to measure, which a checkpoint alone does not carry. It re-runs line matching, gate-target construction, and the quantile/log10 transforms against the raw detector table you point it at, rebuilds the same train/test split (`random_state=42` by default, matching `tools/run_v0_8_real.py`), loads the model, and only then runs the attribution itself. Budget several minutes per source — 4 to 8 for the 3.4 M-event CR file on CPU, longer for a bigger one.

The stage layout (how many encoder/trunk/branch columns are drawn, and where they

Layer	units	share	load	top-20 %	faint
`enc1-enc3`	128/128/64	10.3–10.5 %	1.17–2.39×	39–46 %	0–3 %
`z`	8	7.8 %	14.14×	53 %	0 %
`stem`	64	11.4 %	2.59×	56 %	2 %
`trunk1-3`	128/128/64	6.7–9.2 %	0.93–1.53×	51–57 %	7–14 %
`eb2`	48	1.4 %	**0.42**×	70 %	23 %
`pos1`	128	2.1 %	**0.24**×	60 %	16 %
`pos2`	128	1.9 %	**0.22**×	63 %	20 %
`pos3`	64	1.6 %	**0.37**×	57 %	17 %
`dir1-3`	128/128/64	5.1–5.9 %	0.64–1.15×	52–58 %	14–22 %

Table 1: Per-layer absolute usage, CR reference run, all types pooled. Shaded rows are the ones the tool flags: `load` below 0.5× *and* a real population of faint units. `position_branch` holds 320 units — 22 % of the network's width — and does 5.6 % of its work, while `direction_branch` at identical width does 16.6 %. `z` at 14.14× confirms the latent bottleneck is the hardest-working part of the model.

connect) is derived from the loaded checkpoint's own architecture, not hardcoded to any one source — it renders correctly whether the checkpoint has two particle types and four gate zones (Small/K-40) or six and six (CR).

The v2 counterpart. The AR-flow package has its own circuit for ShowerFlow and CrossingFlow (`MAGIv2/scripts/make_circuit.py`), which draws eight stages (the multiplicity and type heads, the route gate and the five conditional spline flows) with the same Expected Conductance, again against real baselines, and with MLP wires between the most-used units of consecutive layers.

Using these to check how the network works

- **Is the gate physically sane?** Figure 1: a type that cannot produce a given line should route essentially nothing to it. Lines appearing under `mu-` or a neutrino are a red flag.
- **Is the latent bottleneck saturated?** A high `load` on `z` says every latent dimension is working; combined with the KL discussion in Section 2, that is the case for *not* shrinking `latent_dim`.
- **Does one type use the network differently?** Switch buttons and watch which branch brightens. A type whose events are geometrically distinctive should light the position/direction branches more than one that is spectrally distinctive.
- **Is a branch carrying its weight?** Table 1, the `load` column — then an ablation, never the table alone.

5 Primary-conditioned models and grouped generation

This section and Section 6 are advanced. They are needed only when the downstream simulation requires every crossing to carry the identity of the primary that produced it, as in the HENABICS chain, and a user with a single source can skip both.

A model trained on a table with primary truth learns $p(\text{crossing} \mid \text{crossing type}, \text{primary type}, E_{\text{prim}})$ and is sampled per primary.

Data. The 16-column table is the 13-column lineage layout plus `PrimaryTrackId`, `PrimaryPDG` and `PrimaryEnergy`, and `load_detector_table` detects it automatically. The species are simulated and merged separately, so that `EventId` can repeat across them and the group key is `(PrimaryPDG, EventId)`. The number of primaries thrown per species and energy bin, which includes the primaries that left no crossing, is read from the run summaries with `load_run_summaries`.

Training. `build_multiplicity_table` measures $P(k \mid \text{primary type}, E_{\text{prim}}\ \text{bin})$ from the run summaries, with $k = 0$ included, together with the crossing-type probabilities in the same bins. `build_primary_conditioning` appends [onehot(primary type), qt($\log_{10} E_{\text{prim}}$)] to the crossing-type one-hot, so that `cond_dim` $= n_{\text{types}} + n_{\text{primary types}} + 1$, and builds a group id that makes `split_feature_data` split by primary; no shower then has crossings in both the training and the test set. Models trained without primary truth are unaffected, since `cond_dim` defaults to `n_types`.

Generation. `generate_primary_groups` draws the primaries from the GPS macro of the full simulation (`parse_gps_macro` reads its `/gps/hist` spectra), draws k and the crossing types from the multiplicity table, generates the k crossings conditioned on the primary and writes them contiguously under one group key. Primaries with $k = 0$ write nothing and are counted. The command-line entry point is `scripts/generate_geant_source.py -n-primaries N`, with the optional `-species`, `-primary-spectra` and `-key-offset`, while `-n-events` keeps the per-crossing mode of the other models. Crossings that point outward are redrawn with the same conditioning by default (`inward_only=True`), since they cannot enter the detector from the source surface, and the number of outgoing first draws is recorded in the sidecar.

Output. The binary format is `GNTBIN` version 2, a fixed-size record per crossing that starts with the int64 group key, the primary track id, the primary PDG and the primary energy, followed by the fields of version 1; `read_detector_binary` reads both versions. A JSON sidecar records `N_primaries_sampled` per species and the histogram of the sampled primary energies. This histogram is the denominator that normalises the MAGI arm of a comparison with the full simulation, in the same way as the number of primaries thrown normalises the full arm.

Closure of the key. `tools/henabics_closure_dm12.py` verifies that the key, the primary type and the primary energy survive the second Geant4 run (every post-MAGI row joins to exactly one primary, and the carried energy matches to 10^{-5} relative). It then compares the per-primary deposit rate and the joint distribution of deposited and primary energy with the full simulation, per species.

5.1 The v1.0.0 recipe

Six settings of `CVAE_MixEnergy_ContPhi_TaskAdaptive` and of its generation repair the defects exposed by the DM1.2 closure. The run used for the v1.0.0 results is

```
python tools/train_dm12_primtruth.py --uncollided-atom --zone-by-primary \
    --energy-loss-flow --loss-variable logit --first-entry --radius 105 \
    --refit-prior
```

followed by a weighted refit of the prior (Section 7.5).

Uncollided atom (`uncollided_atom=True`). A primary that reaches the surface unscattered crosses it at $E = E_{\text{prim}}$ exactly, which is 53 % of the γ crossings on DM1.2. The gate receives one more component pinned at $\log_{10} E_{\text{prim}}$, allowed only for a crossing of the primary's own species (`add_uncollided_gate_target`), and the quantile transform of E_{prim} is inverted exactly in float64 (`primary_energy_knots`), since an interpolated inversion misses an atom 10^{-4} wide.

Zone table by primary (`-zone-by-primary`). The zone is drawn from a table indexed by crossing type, primary type and one of eight E_{prim} bins (`zone_primary_energy_edges`, `build_zone_probs_table`). The per-type table averaged the γ primaries (55 % uncollided) with the μ^- primaries (none) and generated 36 % uncollided γ, while the table by primary gives 53.2–53.7 % on three seeds against 53.2 % real.

Energy loss flow (`energy_loss_flow=True`). The option requires the uncollided atom and takes `loss_flow_variable="loglog"` or `"logit"`. The continuum crossings of the own species are modelled by a second flow on the energy lost, so that a sampled energy lies below E_{prim} by construction. Without it 6.3 % of the μ^- and 2.0 % of the γ own-species crossings exceeded E_{prim}, and with it none do; the condition-matched AUC of the μ^- primaries goes from 0.672 ± 0.024 to 0.628 ± 0.007 (three seeds).

Weighted prior refit. See Section 7.5.

Latent bank. See Section 7.6.

Multiplicity table (`build_multiplicity_table(..., n_energy_quantiles=65)`). E_{prim} of the primaries that cross is redrawn from quantiles stored per coarse bin and k class, and the crossing types are tabulated on a 0.1 dex grid. The half-decade table placed the crossings evenly inside a bin where $P(k > 0)$ changes steeply, with four times too many muons crossing below 50 MeV, and this alone accounted for most of the excess albedo of the first reinjection arm ($1.50 \rightarrow 1.087$ of the full simulation). `-write-multiplicity-to` writes the table of a run.

6 Tracking surfaces that cut other volumes: first-entry crossings

The method of this section was developed and validated on the DM1.2 cryostat only, and no other mass model has been tested, the Athena one included.

MAGI learns the particles crossing a closed sphere around the region of interest, and the Geant4 run that uses a MAGI sample injects them at that sphere and simulates only the inside. The original scheme requires a sphere that lies entirely in empty space, since a sphere placed as a Geant4 volume overlaps whatever it cuts, and on a real mass model such a position may not exist. Moreover, the particles collected on a sphere in material are not independent of what lies inside it: the albedo returning from the interior is recorded as crossings and is produced again when the interior is simulated, so that a model trained on it counts it twice. The first-entry method removes both problems. It was developed on the DM1.2 geometry (`S1GDML_DM1.2_MultiVerse`, branch `trackG-multiverse` of the Geant4 repository, documented in its `MULTIVERSE.md`); the design is in `docs/TrackG_design_review.md`.

Simulation side. The sphere is not a Geant4 volume. `SteppingAction` tests the straight segment of every step against the sphere and records the crossing from the outside to the inside, and the transport of the mass world is the standard one. A Geant4 parallel world cannot be used, since in Geant4 10.4.3 it switches the whole run to `G4CoupledTransportation`, which changed the secondaries by about a factor of ten (`MULTIVERSE.md`, Section 6). Every track carries a flag that states whether the track or one of its ancestors has been inside the sphere, set at birth for primaries, updated after every step and handed to the secondaries at creation. The crossing file has a 17th column, `AncestorInside`, which is 0 for a first entry (the whole history of the track lies outside the sphere, so it is independent of the interior) and 1 for a returning particle. The radius and the centre are the macro commands `/MultiVerse/radius` and `/MultiVerse/centre`, and one run writes both samples.

MAGI side.

```
df = magi.load_detector_table(filepath="outputDSCryoSphere.dat", sep=r"\s+")   # 17 columns
df = magi.select_first_entry(df)   # keeps AncestorInside == 0, prints the fraction removed
```

`-first-entry -radius <mm>` of the primary-conditioned trainers does this, and the metadata records `crossing_rule` (`first_entry` or `all_crossings`) next to the radius and the centre. A model trained on first entries has to be injected into a run that simulates the whole interior with the same outer geometry and the same radius, and `generate_geant_source.py` writes the positions on the sphere of the saved radius (the check is $|r - R| < 10^{-4}$ mm).

Choosing the sphere. `g0_surface_scan` in the MultiVerse folder samples the surface of a candidate sphere and reports, from the geometry alone, which volumes and materials it cuts and which fraction of the area lies in material; with a list of crossing positions (`points` mode) it gives the fraction of the flux. The sphere may cut passive volumes, it must not cut a sensitive one, and it must enclose the sensitive detector. On DM1.2 the radius of 95 mm is the only one between 30 and 300 mm that is entirely free, while 105 mm cuts a copper plate (2.4 % of the area, 6.9 % of the crossings). The recipe and the closures of this manual use 105 mm, and the test below was run at both radii.

Validation. The test was run on DM1.2 with 72×10^6 muons and 90×10^6 photons at the radii of 95 and 105 mm, and the decisive part uses no model: the first entries of a full run are injected into the same geometry, and the returning entries regenerated by the transport of the injected particles are compared with those of the full run.

Muon primaries	$R = 95$ **mm**	$R = 105$ **mm**
Returning entries, reinjection / full	1.010 ± 0.020	0.962 ± 0.015
Energy deposited inside, reinjection / full	1.004 ± 0.007	0.996 ± 0.007
Triggering primaries of the detector, reinjection / full	645 / 615	596 / 600

The first models produced 16–26 % too many returning entries for the muon primaries. The excess was traced to muons entering below 30 MeV, stopping inside the sphere and decaying, and it is due in part to the multiplicity table and in part to a bias of the learned prior at low E_{prim}. With the v1.0.0 recipe at $R = 105$ mm the numbers are the following.

Muon primaries, MAGI / full	$\alpha = 5$ **prior**	**latent bank**	**reinjection**
Returning entries	1.087 ± 0.016	0.975 ± 0.015	0.962 ± 0.015
Energy deposited inside (keV per primary; full 12.106 ± 0.058)	12.069	12.165 ± 0.058	
Triggering primaries (per 10^6; full 8.33 ± 0.34)	8.86	9.14 ± 0.36	

For the γ primaries the $\alpha = 5$ prior gives an interior deposit of 0.4286 keV per primary against 0.4193 ± 0.0053 (1.2σ with the errors of both runs combined), and the bank gives 0.4451 (3.4σ on the same basis); the bank is therefore used for the muons only. For the muons the bank is chosen for the returning entries (0.975 ± 0.015 against 1.087 ± 0.016) and for the slow-muon fraction. The trigger rate does not support it: the 9.14 triggering primaries per 10^6 lie 2.4σ above the full 8.33 ± 0.34 (the $\alpha = 5$ prior gives 8.86, 1.5σ), counting the error of the full run alone, and neither arm is demonstrated to agree at the detector level.

Limits of the method.

- The crossing state is taken at the pre-step point, which is the convention of the original recording and is exact for the position. Where the sphere cuts thick dense material, the energy lost between the pre-step point and the surface is neglected, and this has not been measured.
- A source lying inside the sphere, such as the activity of a passive layer that is partly inside, is flagged at birth and is not part of the first-entry sample. That part has to be simulated directly in the application run, and this case has not been tested.
- The returning entries are 1–3 % of the crossings in the cases tested, and the detector-level closure (a few hundred triggering primaries, ±5 %) cannot resolve an effect of that size. The test of the rule is thus the count of returning entries and the energy deposited inside.
- The method was validated on the DM1.2 cryostat only, and the Athena mass model, the case that motivated it, has not been tested.

7 Strategies to improve a trained model

The architecture of v1.0.0 is fixed, and the measures below act on the data transforms, on the energy warp, on the prior and on the generation. They are ordered by how early they enter the recipe. Each one states what it does, when it helps and what it costs; none of them is applied silently, and each is reported in the metadata of the run.

Strategy	Acts on	Use it when	Measured effect	Cost
§7.1 Lines out of the CDF warp	energy flow	a pinned line is over-generated	CR 511 keV: 4.84 → 3.40 % of the γ within 50 eV (real 3.45 %)	none
§7.2 Boost ranges of the warp	energy flow	the energy marginal misses a decade	Th warp2 energy RMS 5.20 → 4.83	a tail above the real maximum must be capped
§7.3 Quantile transform on all rows	geometry heads	marginals show ripples at fixed positions	Th232 u_r 3.5 → 1.6, μ ϕ_r 3.7 → 1.3	a refit of a shipped run changes its transformers
§7.4 Prior refit on the frozen encoder	prior	always, as the last stage	CR AUC 0.649 → 0.624	about 35 minutes of laptop CPU
§7.5 Weighted refit	prior	the prior misses a region of the conditioning	DM1.2 slow-muon bias reduced by about two thirds	one more refit
§7.6 Latent bank	generation	the refit is not enough and the bias is declared	DM1.2 slow muons 0.098 → 0.058 (real 0.060)	replays training posteriors, 32 MB
§7.7 Line sampling	generation	a line has to be exactly monoenergetic	no change of the CR score	none

7.1 Exclude the pinned lines from the CDF warp

Geant4 writes a line photon at the exact catalogued energy, so that every pinned line is an atom in the empirical energy CDF. `fit_cdf_warp_knots(energy_y)` then stacks several quantile knots on the atom, and the warp alone maps the probability mass of the line into a sliver a few eV wide: the continuum flow emits a spike at the line before it has learned anything, and an untrained flow already does so. On CryoSphere-CR this made the 511 keV line 1.4 times too strong (4.84 % of the generated γ within 50 eV against 3.45 % real), with the excess carrying continuum-like geometry, and it was the dominant coverage hole of the classifier. Neither the routed loss nor a warm fine-tune removes the spike, while a refit of the warp does.

The line positions are always passed as `fit_cdf_warp_knots(energy_y, ..., exclude_y=line_positions_y)`, and the function warns when it finds an atom. New runs also set `energy_line_routing="supervised"`, so that each event trains only its own component (`argmax(gate_target)`) and the flow never sees a line photon. An existing checkpoint keeps its stored knots, and `tools/refit_energy_flow_posthoc.py` repairs one post hoc (refit warp, flow warm-started, everything upstream frozen), which brought the CR 511 keV fraction to 3.40 % against 3.45 % real. When a line looks over-produced, the first check is whether the stored knots repeat a value of y. The reference trainers `tools/train_cr_v1.py` and `tools/train_iaps.py` pass `exclude_y`, so that a run they produce carries the fix. A run trained before the fix keeps the knots it was saved with until it is repaired, and the run directory does not state which case it is; the knot check is therefore the way to know.

7.2 Boost ranges and the line criterion

The warp of the energy flow is fitted on quantiles of the training energies, and a spectrum that spans several decades can leave a range under-resolved. `tools/train_iaps.py -warp-eps` sets the smallest probability step of the knots, and `-warp-boost LO_MEV,HI_MEV,N` (repeatable) adds N knots inside a range. The SRON Th and Ra models of v1.0.0 use an epsilon of 10^{-6}, boosts of 0.001–0.07 and 0.07–0.12 MeV with 64 knots each, and a generation cap at the real maximum of the energy (3.96112 MeV for Th, 3.18362 MeV for Ra). Two choices of the first attempt proved harmful: a boost between 1.9 and 3.3 MeV placed crossings at 2.7 MeV (2483 per 10^6 for Th,

against 21 real), and an epsilon of 10^{-4} hid the 1–40 keV tail. The line criterion is the fraction of the crossings (0.1 %) and the prominence of the peak is not used, since the latter lost lines (Ra from 9 to 4, Th from 9 to 5); all 26 Ra lines are kept with the fraction criterion.

The cap has to be chosen per source. The energy cap measured on SRON Th (3.96 MeV) applied to every IAPS source produced an apparent per-bin RMS of 17.6 for μ and 8.6 for γ, and the same models without it give 4.3 and 4.9, so the generation cap and the number of events (400 000 per subset) have to be stated whenever an RMS is quoted.

7.3 Fit the quantile transform on all rows

`_fit_quantile_column` previously left the `subsample` argument of the scikit-learn `QuantileTransformer` at its default of 10 000, so that the 10 000 quantiles of every model were computed from 10 000 rows, about one row per quantile. The transformed real data then lay 3 to 4 RMS away from $\mathcal{N}(0, 1)$, while a fit on all rows gives 0.9 to 1.0, and the geometry marginals showed ripples at fixed positions of u_r that had been attributed to the prior. The fit now uses `subsample=len(values)`, and models trained from the release onward carry the correction. The shipped SRON, IAPS and DM1.2 models were trained before it and do not.

The models trained before the correction can be repaired without retraining by swapping the transformers (`tools/refit_geometry_qt.py <tag> <iso>` copies the run `<tag>` to `<tag>_qtfull` and refits the geometry transforms on all training rows; it needs the split of the source, written by `tools/dump_iaps_splits.sh`, and the script is written for the IAPS runs). The swap lowers the per-bin RMS of Th232 u_r from 3.5 to 1.6 and of μ ϕ_r from 3.7 to 1.3, and it has no effect on Ra226, K-40, γ and CR (differences below 0.1). However, the swap is not neutral for every variable: the u_r deviation of μ rises from 4.7 to 5.4, and the 5x detector closures of the swapped Th232 and μ models have not been repeated, so a shipped run is never replaced without that check. The energy is not affected, since it is transformed with $\log_{10}$ and with no quantile map.

7.4 Refit the prior on the frozen encoder

The learned prior of the v0.8 models is trained jointly with the encoder and the decoder, and it does not reach the aggregated posterior it is meant to match. This mismatch is the dominant part of the classifier AUC excess of the generated samples. On CryoSphere-CR, decoding the latents of held-out real events gives an AUC of 0.528 while decoding latents drawn from the prior gives 0.594 with the wide conditioner, and a classifier in latent space can barely separate the two (0.515), so that the decoder amplifies a small latent error into a visible error of the physical variables. A refit of the prior on the frozen encoder lowers the AUC of a retrained model from 0.649 to 0.624 (mean of three training seeds, standard deviation 0.004 between seeds), and it is therefore the last stage of the recipe, after `fit_model` and `save_final_trained_model`:

```
result = magi.refit_prior(
    model, X_cont_train, cond_train, X_cont_val, cond_val,
    checkpoint_dir="trained_models/my_run/refit",   # per-epoch checkpoint, resumable
)
magi.save_refit_model(
    model, result.prior, src_dir="trained_models/my_run", out_dir="trained_models/my_run_final",
    model_name="mix_CR", n_types=n_types, radius=R, hidden=result.hidden,
    n_layers=result.n_layers, val_kl_before=result.val_kl_before,
    val_kl_after=result.val_kl_after, history=result.history)
```

`refit_prior_run` performs both steps for a saved run. The defaults are the measured recipe, a conditioner of 128×128 and six coupling layers, a fresh initialisation and 60 epochs at 10^{-3} with cosine decay, and the weights of the epoch with the lowest validation KL are kept. The written run directory is identical to the source except for the prior, and its metadata records the refit with the KL before and after. `prior_posterior_auc` gives the diagnostic (0.5 means that the prior reproduces the aggregated posterior), which was 0.51–0.52 after the refit on three seeds. A refit

takes about 35 minutes on a laptop CPU for CR and continues after an interruption when the same call is repeated. `tools/train_cr_v1.py -refit-prior` runs the whole recipe for CryoSphere-CR, and `tools/train_dm12_primtruth.py -refit-prior` does so for the primary-conditioned models, where the refit brings the μ^- AUC of the uncollided-atom model from 0.734 back to 0.695.

7.5 Weighted refit

`sample_weight_train` and `sample_weight_val` weight the events of the refit, to pull the prior toward a region of the conditioning where it misses the aggregated posterior. On DM1.2 the learned prior over-produces slow muons for the μ^- primaries below 100 MeV, which then stop and decay inside the sphere, and weighting the events of those primaries by $\alpha = 5$ removes about two thirds of the bias (`tools/dm12_weighted_refit.py <run> <out> <alpha> [E_cut]`). The value $\alpha = 5$ is the reference because it keeps the detector level, and $\alpha = 20$ is a sensitivity test.

7.6 Latent bank

`magi.build_latent_bank` (or `tools/build_latent_bank.py <run> <split npz> [K]`) stores the posterior means and variances of the training split with the model, and `generate_geant_source.py -latent-bank <npz>` then draws z from the posterior $q(z \mid x')$ of a training event x' with the same crossing type, the same primary type and one of the $K = 50$ nearest values of $\mathrm{qt}(E_{\mathrm{prim}})$, in place of the prior. The heads still draw their own noise, so no training crossing is reproduced. However, the bank is a replay of training posteriors: it gives up the interpolation of $p(z \mid \mathrm{cond})$ between training conditionings, it occupies 32 MB on DM1.2 (522 642 events), and rows whose conditioning group is absent fall back to the prior. It is available in group mode only. On DM1.2 it brings the fraction of slow muons ($E < 10$ MeV, $E_{\mathrm{prim}} < 100$ MeV) from 0.098 to 0.058 against 0.060 real, while for the γ primaries it hardens the crossings (interior deposit +6 %, 3.4σ) and the $\alpha = 5$ prior is used instead. The bank masks a failing prior and does not repair it, and a result obtained with it has to be declared as such.

7.7 Line sampling and the inward condition

`line_sampling="delta"` (`-line-sampling delta` in `scripts/generate_geant_source.py`) emits the line events at exactly the line energy. It did not improve the classifier score on CR, and the default remains `"gaussian"`.

The direction head models the global variables u_v and ϕ_v, and whether a crossing points inward therefore depends on its position on the sphere, which the head does not enforce. After the prior refit 1.4 % of the generated CR crossings point outward, against 13–15 % on DM1.2 before the redraw. `inward_only=True` in `generate_primary_groups` redraws them with the conditioning fixed. The alternative is `geometry_transform="quantile_u_r_mu_phi_r_psi"` (`-local-direction`), which models the direction in the local frame of the crossing point with $\mu = -\hat{r} \cdot v$, the cosine with the inward normal, and ψ, the azimuth around it, so that every generated crossing is inward by construction. On DM1.2 this option costs realism outside the seed spread (AUC μ^- 0.656 against 0.629–0.634, γ 0.688 against 0.678–0.683), because the coupling of the energy to the zenith is lost (A3 in `docs/CVAE_vs_AutoRegFlow.md`), and v1.0.0 keeps the global frame with the redraw.

8 The tools

Scripts in `tools/` operate around a run rather than inside one. All of them force CPU execution before importing `magi`. Run them from the repository root.

8.1 Training and scoring

`run_v0_8_real.py` A full training run on the real sources with live per-epoch logging and chunked generation, so a long run is observable and OOM-safe. Saves a checkpoint, spectra and a recovery report per source.

```
python tools/run_v0_8_real.py --sources CR Small --run-tag v0_8_2 \
    --epochs 40 --seed 42 --prior-zone-conditioning
```

Flags: `-sources`, `-run-tag`, `-epochs`, `-seed`, `-n-gen`, `-gen-chunk`, `-prior-zone-conditioning`, `-gate-target-bandwidth-mode` (default `resolution`), `-continuum-flow-bins`, `-warp-boost-cr`.

`acceptance_v0_8.py` The pass/fail harness. It grades a trained run against explicit thresholds, so an 80-minute run is scored objectively rather than by eye. The same numbers are the paper's validation table.

```
python tools/acceptance_v0_8.py --sources CR Small --seeds 42 7 13 \
    --json docs/_data/acceptance.json
```

It reports four things: per-line integral recovery (resolution-scaled $\pm 5\sigma$ window, local continuum subtracted, normalized for $N_{\mathrm{real}}/N_{\mathrm{gen}}$); near-line continuum ratio, i.e. whether the gate dug a hole beside each line; the energy-marginal Wasserstein distance, global and per decade band; and the coupling residuals.

`score_v0_7_2.py` Scores a v0.7.2 categorical-head run on the same axes, so the two heads can be compared like with like. It reads each checkpoint's own `preprocessing_metadata` rather than re-deriving it.

8.2 Audits

`line_centroid_audit.py` For each matched line, histograms the real spectrum locally at eV resolution, fits the centroid and reports |catalogue − measured| in FWHM units, failing loudly above one FWHM. Run this before committing to a long training run: a mispositioned line cannot be fixed by any amount of training.

`build_candidate_lines_from_geant4.py` Builds the candidate-line table from a GDML mass model and Geant4's fluorescence data. It takes transition *labels* from the Bearden table (only that directory carries them) and transition *energies* from the EADL table, because EADL is the emission for the `SpacePhysicsList` setup.

`gate_posterior_prior_audit.py` Localizes a per-line intensity failure to *where in the pipeline* it happens: the gate itself, or the latent distribution the gate is fed at generation time.

The gate is conditioned on z (`energy_flow_condition="z_cond"`). At training and reconstruction time it sees encoder codes, $z \sim q(z \mid x_{\mathrm{real}})$; at generation time there is no real event, so it sees prior samples, $z \sim p(z \mid c)$. Section 2 argues those two differ substantially. This script measures whether that difference actually moves the gate's output, which is the step between "the latents differ" and "the line intensities are wrong".

It is training-free: it loads a checkpoint and runs two forward passes over the real dataset, one with encoder codes and one with prior samples. For every particle type × gate slot (continuum components plus each pinned line) it prints the mean gate probability under each, and their **prior/posterior ratio**:

```
python tools/gate_posterior_prior_audit.py --sources CR Small \
    --max-events-per-type 200000
```

```
type      slot                    post       prior   prior/post
gamma     continuum_0       9.3753e-01  9.4490e-01        1.008
gamma     Al K-alpha1       5.7883e-03  1.7573e-03        0.304
mu-       Cu K-alpha1       1.5495e-11  1.1889e-08      767.271
```

Read the ratio column:

- ≈ 1 — the gate behaves the same whether it is fed encoder codes or prior samples. Any line error for that slot is the gate's own calibration, and gate-side fixes are the right place to look.
- $\ll 1$ — the line is routed *less* often at generation time than during training. The gamma rows above show this: Al Kα_1 at 0.304 means generation reaches that component roughly a third as often as reconstruction does, and no amount of gate reweighting during training will show up, because the training-time gate was already correct.
- $\gg 1$ — over-routing at generation time. The extreme values in the `mu-` rows are an artifact of dividing two numbers around 10^{-11}–10^{-8}: for a type that physically never produces that line, both fractions are numerically zero and the ratio is meaningless. Check the absolute columns before believing a large ratio.

A widespread departure from 1 is evidence that the *prior* is the thing to change — which is what motivated `prior_zone_conditioning` (§2)

`memorisation_test.py` Does MAGI generate, or does it recall its training data? A large amplification factor (§11.4.2) does not by itself distinguish a generative model from a kernel-density resampler, which turns the same crossings into an unlimited sample by construction without learning anything. This tool is the check.

For each generated sample it measures the nearest-neighbour distance to a held-out reference set of real crossings, in the model's own feature space (azimuths carried as $(\cos, \sin)$ pairs, so the metric respects periodicity). A model that memorises places generated samples closer to training points than real data lies from itself; a model that generalises places them at the same distance real data sits from itself.

```
python tools/memorisation_test.py cr      # or: dm12
```

The metric is calibrated in both directions before it is trusted: a held-out real split must read as real (ratio ≈ 1), and a battery of synthetic "memorisers" — literal training points at increasing jitter — must read as recall (ratio $\to 0$, near-duplicate fraction $\to 100\,\%$). The script refuses to print a verdict if either control fails. Read the reported ratio as ≈ 1 generalises, $\ll 1$ memorises, $\gg 1$ generates off the data manifold — see §11.4.2 for the CR and DM1.2 results.

The dataset paths are placeholders

`DATASETS` at the top of the script hardcodes a real and a generated-sample path per source (`"cr"`, `"dm12"`). Point them at your own training file and a text/binary file of MAGI output (the `generate_detector_table_to_file` output, §4) before running it.

8.3 Synthetic stress tests

`synthetic_stress_test_coupled.py`, `_cr_multimodal.py` and `_small_singlepeak.py` build synthetic data with a known answer — an injected energy–geometry coupling, a rare-line cluster, a sparse tail. Use them to gate a change *before* spending a real run on it; they finish in minutes.

8.4 Unit tests

```
python -m pytest MAGI_package/tests/ -q
```

238 tests covering flow round-trip identity, checkpoint save/load config matching, gate-target construction (including the exact/resolution A/B), prior zone-conditioning, the two visualization tools (§4.10), and public API docstring coverage. They catch mechanical regressions; they do not tell you a trained model reproduced your source.

9 Example: a full run

This is a complete run on the reference cosmic-ray source, from raw data to a Geant4 source file. On a CPU-only Apple Silicon machine it takes roughly 90 minutes for 40 epochs plus generation.

9.1 The short path

Everything below is what `tools/run_v0_8_real.py` does. In practice:

```
export CUDA_VISIBLE_DEVICES=-1
python tools/run_v0_8_real.py --sources CR --run-tag my_run \
    --epochs 40 --seed 42 --prior-zone-conditioning \
    2>&1 | tee logs/my_run_CR.log
```

9.2 The long path, step by step

```
import os
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"
import numpy as np, joblib, magi

magi.initialize_environment(seed=42, cpu_only=True)

# --- 1. load ------------------------------------------------------
df = magi.load_detector_table("TrainingData/alloutputDSCryoSphereCR.dat")
magi.report_basic_table_checks(df)

# --- 2. physical features -----------------------------------------
prep = magi.build_physical_features(
    df, center=(0.0, 0.0, -507.66), radius=100.0, source_type="cosmic_ray")
magi.print_physical_summary(prep)

# --- 3. lines: detect on the RAW energies, before the feature frame --
candidate_lines = magi.load_candidate_energy_lines(
    "CandidateLines/cryosphere_lines.json")["lines"]
E = prep["features"]["Energy"].to_numpy()

res = magi.detect_energy_lines(
    E, binning_mode="log_fixed_count", n_bins=1024,
    prominence_factor=3.0, window=5, candidate_lines=candidate_lines,
    refine_bin_width_mev=4.0e-6)
matched = [m for m in res["matched_lines"] if m["count"] >= 100]
matched += magi.confirm_unresolved_candidate_lines(
    E, candidate_lines, matched, resolution_ev=4.0)
magi.print_detected_energy_lines(res)

# --- 4. feature frame ---------------------------------------------
feature_pack = magi.build_feature_dataframe(
    prep, energy_binning_mode="log_fixed_count", n_bins=512, min_counts=20,
    geometry_transform="quantile_u_r_u_v_phi_r_phi_v",
    energy_transform="log10")          # the v0.8 head needs y = log10(E)
magi.report_feature_dataframe(feature_pack)

E_full = feature_pack["filtered_prep"]["features"]["Energy"].to_numpy()
```

```
gate_targets = magi.build_gate_targets(
    E_full, feature_pack["energy_bins"], matched,
    bandwidth_mode="resolution", bandwidth_fwhm_mev=4.0e-6)

line_positions_y = np.log10(
    [m["candidate_energy_mev"] for m in matched]).astype(np.float32)
line_logsigma = magi.line_logsigma_from_resolution(
    10.0 ** line_positions_y, resolution_ev=4.0, fwhm=True)

# --- 5. dataset: the gate targets ride along as extra feature columns -
feat = feature_pack["feat"].copy()
for j in range(gate_targets.shape[1]):
    feat[f"gate_target_{j}"] = gate_targets[:, j]
cont_cols = ("u_r_q", "u_v_q", "phi_r_q", "phi_v_q", "energy_y") + tuple(
    f"gate_target_{j}" for j in range(gate_targets.shape[1]))

dataset_pack   = magi.filter_particle_types_continuous_geometry(
                     feat=feat, prob_threshold=1e-5, cont_cols=cont_cols)
split_pack     = magi.split_feature_data(dataset_pack, random_state=42)
magi.report_split_summary(split_pack, dataset_pack["n_types"])
scaled_pack    = magi.scale_continuous_features(split_pack, scale_cols=())
condition_pack = magi.build_conditioning_and_weights(
                     scaled_pack, dataset_pack["n_types"],
                     idx_to_type=dataset_pack["idx_to_type"], alpha=0.5)
ds_pack        = magi.build_tf_datasets(condition_pack, batch_size=4096)
magi.report_tf_datasets(ds_pack)

# --- 6. per-type zone probabilities for the conditioned prior --------
n_zones   = gate_targets.shape[1]
zone_cols = dataset_pack["X_cont_raw"][:, -n_zones:]
zone_probs = np.zeros((dataset_pack["n_types"], n_zones))
for t in range(dataset_pack["n_types"]):
    mask = dataset_pack["y_type"] == t
    row  = zone_cols[mask].mean(axis=0) if mask.any() else np.zeros(n_zones)
    s    = row.sum()
    zone_probs[t] = row / s if s > 0 else np.eye(n_zones)[0]

# --- 7. model and training -------------------------------------------
model = magi.CVAE_MixEnergy_ContPhi_TaskAdaptive(
    n_types=dataset_pack["n_types"], latent_dim=8,
    line_positions_y=line_positions_y, line_logsigma_init=line_logsigma,
    continuum_mode="flow", continuum_flow_warp="cdf",
    energy_flow_condition="z_cond",
    prior="coupling", prior_zone_conditioning=True, zone_probs=zone_probs,
    gate_focal_gamma=1.0, w_gate_aux=2.0)

magi.compile_model(model, learning_rate=2e-4)
magi.print_model_structure(model)

history = magi.fit_model(
    model, ds_pack["train_ds"], ds_pack["val_ds"],
    epochs=40, callbacks=magi.build_default_callbacks(), verbose=2)

# --- 8. checkpoint ---------------------------------------------------
magi.save_final_trained_model(
    model=model, save_dir="trained_models/my_run", model_name="mix_CR",
    history=history, model_config=model.to_generation_config(),
    preprocessing_metadata=preprocessing_metadata,
    normalization_metadata=prep["normalization"])
```

9.3 Producing the Geant4 source file

```
python scripts/generate_geant_source.py \
    --save-dir trained_models/my_run \
    --model-name mix_CR \
    --metadata-file trained_models/my_run/mix_CR_metadata.json \
    --output-file generated/cr_source.dat \
    --n-events 10000000
```

The file format. One particle per line, whitespace-separated:

```
ParticleName Energy X Y Z Vx Vy Vz

gamma 0.131632939 -35.3167992 -93.4576950 -503.372406 \
      0.676084936 0.689519465 -0.259753942
e-    0.542123100  12.3456000  45.1234000 -510.332100 \
     -0.200000000 0.300000000 -0.932000000
```

Energy in MeV, position in mm, direction a normalized Cartesian unit vector. `ParticleName` must match a Geant4 particle definition exactly (`gamma`, `e-`, `proton`, `alpha`, `mu-`, `mu+`, ...) — an unrecognized name is the usual cause of "Unknown particle name" at load. The binary format (`-format binary`) carries the same fields.

What the Geant4 side must provide. Consuming this needs two classes in the application: a `PrimaryGeneratorAction` that can read the file and emit one primary per event instead of using `G4GeneralParticleSource`, and a `PrimaryGeneratorMessenger` exposing that as macro commands. You can find examples in the `Geant4_src` folder with a specific `README` file. In the companion project the messenger declares thirteen commands in the `/generator/` directory:

Command	Purpose
Static file mode — read a pre-generated source	
`/generator/generatedFile`	Path to the source file. Loading it also *activates* file mode, so the next command is usually unnecessary.
`/generator/useGeneratedFile`	Enable/disable file mode explicitly.
`/generator/generatedFormat`	`text` or `binary`.
`/generator/binaryBufferSize`	Particles held in memory per read.
Auto mode — Geant4 invokes MAGI itself	
`/generator/autoGenerateMLInput`	Run the generator before loading.
`/generator/mlPython`	Python interpreter with `magi` installed.
`/generator/mlScript`	Path to `scripts/generate_geant_source.py`.
`/generator/mlModelDir`	Trained-run directory.
`/generator/mlModelName`	Checkpoint filename stem.
`/generator/mlMetadataFile`	The run's `*_metadata.json`.
`/generator/mlNumParticles`	Particles to generate.
`/generator/mlSeed`	Seed passed through to MAGI.
`/generator/mlChunkSize`	Generation chunk size.

In auto mode the generator shells out to exactly the command shown at the top of this subsection, assembled from those values:

```
"$mlPython" "$mlScript" --save-dir "$mlModelDir" \
    --model-name "$mlModelName" --metadata-file "$mlMetadataFile" \
    --output-file "$generatedFile" --n-events $mlNumParticles \
    --seed $mlSeed --chunk-size $mlChunkSize --format {text|binary}
```

so the C++ side needs no knowledge of MAGI beyond this argument list, and the same script is testable standalone from a shell.

A complete macro. This is the production configuration from the companion project's `try.mac`, trimmed to the generator part:

```
/run/initialize
/random/setSeeds 175 2024

/generator/autoGenerateMLInput true
/generator/generatedFormat     binary
/generator/binaryBufferSize    100000

/generator/mlPython   /path/to/envs/tf-metal/bin/python
/generator/mlScript   /path/to/MAGI/scripts/generate_geant_source.py
/generator/mlModelDir /path/to/MAGI/trained_models/my_run
/generator/mlModelName   mix_CR
/generator/mlMetadataFile /path/to/MAGI/trained_models/my_run/mix_CR_metadata.json

/generator/mlNumParticles 100000000
/generator/mlSeed         1752024
/generator/mlChunkSize    100000

/generator/generatedFile /path/to/tmp/geant_input_seed_1752024_CR.gntbin

/runAction/setTotalEvents 100000000
/run/beamOn               100000000
```

Running these in parallel

Give every job its own Geant4 seed (`/random/setSeeds`), its own MAGI seed (`/generator/mlSeed`) *and* its own `/generator/generatedFile` path. Sharing any of the three across concurrent jobs either duplicates the particle stream or has two processes writing one file. Prefer `binary` and auto-generation for large campaigns: particles are held in memory, so 10^8 in text form is heavy.

10 Example: a validation

10.1 The short path

```
python tools/acceptance_v0_8.py --sources CR --run-tag my_run \
    --seeds 42 7 13 --json docs/_data/my_run_acceptance.json
```

10.2 What it checks, and against what

Criterion	Threshold	Notes
Per-line recovery	[0.8, 1.2], two-sided	Only for lines above the detection floor. Two-sided because over-generation is the dominant failure.
Per-line stability	$\sigma \leq 0.15$ over ≥ 3 seeds	So a user's single unattended run likely lands in band.
Detection floor	$\geq 5\sigma$ Poisson	The particle-physics discovery convention. Below-floor lines are *not modelled*, and their absence is a correct result.
Near-line continuum	≥ 0.85	Did the gate dig a hole beside the line?
Energy marginal	Wasserstein ≤ 0.05	Global, on $\log_{10} E$.
Coupling	$\max \lvert\Delta\text{corr}\rvert \leq 0.05$	Over $(\log E, u_r, u_v, \phi_r, \phi_v)$.

10.3 Doing it by hand

```
real = magi.reconstruct_real_test_physics(
    split_pack["X_cont_test"], split_pack["E_test_raw"],
    center=(0.0, 0.0, -507.66), radius=100.0,
    geometry_metadata=geometry_metadata)

# Generate as many events as there are real ones - see the warning in 3.9.
gen_pack = magi.generate_latent_outputs(
    model, n_samples=real["E"].size, type_probs=type_probs,
    n_types=n_types, idx_to_type=idx_to_type)
reco = magi.reconstruct_generated_features(
    gen_pack, energy_head_mode="mixture",
    geometry_metadata=geometry_metadata, energy_metadata=energy_metadata)
gen  = magi.reconstruct_generated_physics(reco, center=(0,0,-507.66), radius=100.0)

# 1. marginals
scores = magi.compute_wasserstein_scores(real, gen)
print(scores["logE"])

# 2. geometric constraints that must hold by construction
magi.report_generated_constraints(gen, radius=100.0)
magi.report_final_ranges(real, gen)

# 3. per-line recovery
recovery = magi.compute_line_integral_recovery(
    real["E"], gen["E"], matched, energy_bins,
    energy_component_idx_gen=reco["energy_component_idx_gen"],
    resolution_ev=4.0, neighbour_lines=candidate_lines)
for r in recovery:
    print(f"{r['label']:20s} {r['recovery_ratio']:6.3f}  "
          f"sig={r['line_significance']:7.1f}")

# 4. the joint distribution - this is the one that matters
cols = ["logE", "u_r", "u_v", "phi_r", "phi_v"]
df_real, df_gen, df_both = magi.build_real_generated_featureframes(real, gen)
_, corr_real = magi.plot_correlation_matrix(df_real, cols, show=False)
_, corr_gen  = magi.plot_correlation_matrix(df_gen,  cols, show=False)
print("coupling residual:", np.abs(corr_gen - corr_real).to_numpy().max())

magi.compare_hist_with_residuals(real["E"], gen["E"], "Energy [MeV]",
                                 bins=200, ratio_clip=1.0)
```

10.4 Reading the output

- **Coupling residual** is the headline. Below 0.05 means the joint distribution is reproduced, which is what MAGI is for.
- **Wasserstein on** $\log_{10} E$ below 0.05 means the spectrum's shape is right.
- `line_significance` below 5 means the line is not significantly detected in the real data either; its recovery ratio is noise and should be ignored, not reported as a failure.
- `sideband_contaminated` true means a neighbouring line sits in the continuum side-bands, so the continuum subtraction for that line is unreliable.
- A statistical floor of $\sqrt{1/n_{\text{real}} + 1/n_{\text{gen}}}$ bounds how precisely any recovery ratio can be known. For a line with a few hundred events this is already several per cent.

11 Accuracy: what has actually been measured

Two different questions get answered here, at two different costs. The **crossing-level** numbers (§11.4.3, §11.4.4) come from `tools/acceptance_v0_8.py`: a trained checkpoint graded against a

held-out real sample in MAGI's own feature space, in minutes. The **detector-level** numbers (§11.4.1) are the expensive, decisive check: MAGI crossings are injected back into the real Geant4 mass model and the resulting detector spectrum is compared against a full simulation or a matched sample of real recorded crossings. Crossing-level agreement does not guarantee detector-level agreement — a joint distortion invisible to any marginal can still move a downstream rate — so trust the detector-level numbers first.

The measured state of v1.0.0 is in Sections 11.1, 11.2 and 11.3. The record of the v0.8.2 beta follows in Section 11.4 and is marked superseded where v1.0.0 changes it.

11.1 Classifier two-sample test

The figure of merit used to compare models is the AUC of a gradient-boosting classifier that separates real from generated crossings (`tools/score_prior_refit.py`; features $\log_{10} E$, the geometry variables and the particle type). An AUC of 0.5 means that the classifier cannot tell the two samples apart, and the value obtained for two disjoint random subsets of the real data is the floor given by sampling noise. The Wasserstein distances and the correlation residuals of the acceptance harness remain useful to locate a defect, whereas the AUC is the single number that responds to any difference of the joint distribution. Three points of protocol decide whether a number can be quoted.

- **Random real subsets and an interval.** `tools/auc_intervals.py` draws, for each replication, an independent generation seed, an independent random subset of the real crossings and an independent classifier split, and reports the mean and the standard deviation, the paired difference between models and a real-versus-real null. The first N rows of a raw file are not a random sample of it: on CryoSphere-CR they contain a single shower of 1364 crossings at ten times its share of the file. With ten replications on 400 000 crossings the null is 0.5004 ± 0.0012, and two single-run AUCs are considered different only when they differ by more than approximately 0.008 (3σ of a difference).
- **Folds grouped by shower** whenever the data have showers. The crossings of one primary are correlated, and with row-level folds the classifier recognises individual showers: on DM1.2 two real samples give 0.54 for μ^- against 0.50 with folds grouped by primary (`tools/dm12_condmatched_auc.py`).
- **Matched conditioning** for the primary-conditioned task, generating one crossing for every real test crossing with the same crossing type, primary type and primary energy.

The intervals contain the sampling and the classifier variance only. The variance between independent trainings is measured by retraining at different seeds (`tools/train_cr_v1.py -seed`), and an 80-epoch retraining on three seeds lowers the standard deviation of the AUC between seeds from 0.0042 to 0.0020. The mean AUC does not improve (0.6244 against 0.6256), and the $\log_{10} E$ per-bin RMS rises from 2.60 to 3.10, so that the seed term quoted in this manual is the 0.0042 of the shipped setting.

CryoSphere-CR model	AUC (10 replications)	inward only
v0.8.2, original	0.6613 ± 0.0018	0.6376 ± 0.0043
+ wide prior refit and line-free warp (v0.9.4)	0.6191 ± 0.0022	0.6156 ± 0.0025
retrained from scratch with the fixes built in (seed 42)	0.6490 ± 0.0015	0.6279 ± 0.0032
the same with the prior refit on the frozen encoder, seed 42	0.6270 ± 0.0025	0.6180 ± 0.0025
the same, seed 7	0.6196 ± 0.0026	0.6145 ± 0.0025
the same, seed 13	0.6267 ± 0.0031	0.6159 ± 0.0025
real versus real	0.5004 ± 0.0012	0.5010 ± 0.0007

The retraining with the fixes alone moves the AUC from 0.6613 to 0.6490, and the prior refit then brings the three seeds to 0.6196–0.6270 (mean 0.624, standard deviation 0.004). The residual excess over the null of 0.5004 is therefore approximately 0.12 on all crossings and approximately

0.11 on the inward ones, and it is due to the prior together with the per-variable heads (Section 12). Overall, the classifier separates the generated from the real crossings of CR clearly better than two real subsets, and the generated sample is therefore not indistinguishable from the real one, while the AUC of the repaired models is lower than that of the original model by approximately 0.04.

11.2 Per-bin deviation of the marginals

`magi.marginal_bin_rms(real, gen)` gives the Poisson-weighted RMS deviation of a generated marginal from the real one over 50 equal-count bins of the real sample; a value of 1 means agreement within counting noise, and the real-versus-real null is 1.00 ± 0.03. Exact atoms have to be left out of the continuum marginal through `exclude_windows`, due to the spurious deviation of tens of sigma that a bin edge on a line produces by splitting its slightly smeared generated copy, and the lines are scored by the line recovery instead. `tools/marginal_rms_intervals.py` evaluates it with the replications of the AUC protocol. On CryoSphere-CR (± 0.01 in $\log_{10} E$ removed around each pinned line):

CryoSphere-CR model	$\log_{10} E$	u_r	u_v	ϕ_r, ϕ_v	**mean of seven**
v0.8.2, original	4.75 ± 0.08	4.93	4.87	4.25–5.40	4.90 ± 0.06
+ wide prior refit and line-free warp	3.43 ± 0.10	4.33	4.67	4.08–4.66	4.30 ± 0.06
retrained with the fixes and the prior refit (3 seeds)	2.33–3.09	4.46–4.48	4.48–4.90	4.13–4.77	4.23–4.28
real versus real	1.01	1.05	0.97	0.97–1.01	1.00 ± 0.03

The energy marginal improves from 4.75 to 2.3–3.1 with the fixes, while the geometry marginals stay at 4.1–4.9 on every model, and the deviation that remains therefore lies in the geometry and is the same for the original, the repaired and the retrained models. On the IAPS Triple sources, generated without the SRON cap, the mean per-bin RMS is 4.2 for K-40, 4.3 for μ, 4.9 for γ, 5.3 for Ra and 5.5 for Th, at the level of CR and SRON, with a null of 0.9 to 1.1. For μ the residual comes from the prior (decoding the posterior gives 1 to 1.5, the prior 2.9 to 3.8) and, to a smaller extent, from the quantile transform of Section 7.3. Overall, the mean per-bin RMS of every source lies at four to five times the counting noise, and the energy marginal of CR is the only one that the fixes brought below that level.

11.3 Detector-level closure

The generated crossings are injected into the downstream Geant4 run, and the rate of events per detector is compared with the run driven by the real crossings, through the ratio R defined in Section 11.4.1. Two caveats have to be quoted with every number of this subsection. The real arm of the SRON CR closure re-uses crossings that belong to the training sample, which makes the test dependent on the training data, and the v1.0.0 fixes are bundled in one retraining; the closure therefore does not attribute the change to any single one of them.

On SRON CR (84 chunks, 6.0×10^6 MAGI against 6.0×10^6 real crossings) the ratio is $R = 1.006 \pm 0.028$ over all energies, 1.073 ± 0.082 for 0.1–30 keV, 0.979 ± 0.082 for 1–20 keV and 1.039 ± 0.117 for 1–7 MIP. The deficit that the beta had measured ($R = 0.846 \pm 0.088$, Section 11.4.1) closed on 14 October and is not reproduced. The closure spectrum between 0.1 keV and 1 MeV agrees with the full simulation ($\chi^2 = 28.8$ for 23 bins, Kolmogorov–Smirnov $D = 0.039$, $p = 0.22$).

On IAPS Triple, injecting five times the statistics of each source gives $R = 0.97 \pm 0.02$ pooled over the five sources, computed as the equal-weight sum of the per-primary rates, since a sum of the counts over the primaries gives a wrong 0.18. The Th and Ra closures at the two laboratories are the weakest numbers of the validation and disagree in sign.

Source	Set-up	Model	R	Reading
Th	IAPS Triple, 5x	IAPS model, SRON cap not applied	0.88	2.4σ low
Ra	IAPS Triple, 5x	IAPS model, SRON cap not applied	0.86	2.8σ low
K-40, γ, μ	IAPS Triple, 5x	IAPS models	0.97 each	consistent with 1
Th	SRON, warp2 model	boosted CDF warp, capped at the real maximum	1.40 ± 0.22	high
Ra	SRON, warp model	boosted CDF warp, capped at the real maximum	1.31 ± 0.19	high

The two pairs differ in the detector, in the mass model and in the energy model (the SRON warp models carry the tail defects of Section 12), so that they are not repeated measurements of one quantity, and no common cause of the opposite signs has been established. Until it is, a detector-level rate obtained from a Th or Ra source is to be quoted with the deviation of the table, which is 12–14 % low at IAPS and 31–40 % high at SRON. Overall, the closures of CR and of the K-40, γ and μ sources of IAPS agree with the full simulation within their uncertainties, while the Th and Ra sources do not and are declared.

The first-entry closures on DM1.2 are in Section 6, together with the choice between the $\alpha = 5$ prior and the latent bank for the muons.

11.4 History: the measurements of v0.8.2

Superseded where marked

The subsections that follow are the validation record of the v0.8.2 beta. They are kept because the DM1.2 closure, the memorisation test and the coupling experiment are independent of the later fixes and have not been repeated, and each result that v1.0.0 changes is marked as such. In particular, the deficit of SRON CR ($R = 0.846 \pm 0.088$) was closed on 14 October and is replaced by Section 11.3.

11.4.1 Detector-level validation: DM1.2 and SRON (v0.8.2)

Two independent mass models have been closed this way, on three source configurations. The comparison used throughout is the normalization-free ratio

$$R = \frac{N^{\mathrm{MAGI}}_{\mathrm{events}} / N^{\mathrm{MAGI}}_{\mathrm{crossings}}}{N^{\mathrm{real}}_{\mathrm{events}} / N^{\mathrm{real}}_{\mathrm{crossings}}},$$

counted per injected crossing on both sides, through the identical mass model, binary, physics list and scoring code. R needs no exposure, no detector area and no crossing efficiency χ — it cancels every quantity that has historically caused bookkeeping mismatches between the MAGI side and the Geant4 side. $R = 1$ is a perfect surrogate.

DM1.2 — INAF laboratory cryostat, cosmic-ray muons. The cleanest result available: a 3-seed ensemble (1.2×10^6 crossings generated in total) against 2×10^8 muons thrown isotropically through the full mass model.

Band	Reference	MAGI	R
all deposits	1,733	2,019	0.970 ± 0.032
100–300 keV (MIP)	1,325	1,589	$\mathbf{0.999 \pm 0.037}$
300–1000 keV	310	323	0.868 ± 0.069
30–100 keV	42	46	0.912 ± 0.195
0.1–30 keV	41	43	0.873 ± 0.191
>1000 keV	13	17	1.089 ± 0.401

Crossing coverage is 100 % and species fractions are correct to 0.3 %. The MIP band carries the best statistics and lands almost exactly on the ideal value; the 0.1–30 keV row is reference-limited (41 events) rather than MAGI-limited, and more MAGI generation cannot tighten it — nor should it be read as an X-IFU-band result, since DM1.2 is an anti-coincidence detector, not a TES absorber. DM1.2 has a single sensitive detector, so its uncertainties are pure Poisson: there is no across-detector spread to quote.

A real, disclosed defect remains: a coherent ∼4 % *downward* shift in deposited energy (Kolmogorov–Smirnov $D = 0.056$, $p = 0.006$), traced to incidence angle — MAGI's detector-aimed tracks sit slightly too close to normal incidence (median $\sec\theta$ 0.955 of the reference) — which in turn comes from a direction-conditional species-mix error: at small impact parameter MAGI runs +9 % muons and −14 % electrons against a global species mix correct to 0.3 %. Per-species energies are unaffected.

SRON — X-IFU-like flight model, cosmic rays. Measured two ways. Against real recorded crossings, injected back to back with MAGI-generated ones through the identical geometry (∼3.6 M crossings per arm, production checkpoint `v0_8_2_CR_ingoingfix`):

Band	R
all deposits	0.920 ± 0.034
0.1–30 keV (X-IFU band)	$\mathbf{0.846 \pm 0.088}$
1–20 keV	0.767 ± 0.087
1–7 keV (MIP)	0.807 ± 0.124

The same checkpoint, scored per detector against a full six-detector simulation, gives a mean flux ratio of **0.79** (0.57–0.90 across the six) with a spectral shape *indistinguishable* from the reference (Kolmogorov–Smirnov $p = 0.20$). Normalizing the same per-detector data by the crossing efficiency χ gives $R = 0.76$ — consistent with the table above by three independent code paths.

Superseded: this deficit closed on 14 October at $R = 1.006 \pm 0.028$ (Section 11.3), and the paragraph is kept as the record of the beta. The residual ∼15–24 % deficit was then only partly explained. 3.6 % of it traces to an inconsistent ingoing-particle cut: the SRON training sets were recorded before the simulation started excluding outgoing crossings, so they — and MAGI's output with them — contain ∼10–14 % outgoing particles that a corrected training set does not. Correcting it and retraining moved Detector 1's 0–20 keV ratio from 0.802 ± 0.078 to 0.941 ± 0.088. The remainder is not a broken marginal: a classifier trained to separate real from generated crossings reaches AUC 0.68 (0.5 would be indistinguishable) with the separating signal spread evenly across every continuous feature — a diffuse joint mismatch, not one wrong variable — and reweighting the generated sample to match the real joint by likelihood ratio does not repair it (the reweighted sample stays separable, with effective sample size collapsing under the attempt). See §11.4.6 for the one architectural fix tried against this so far.

SRON — ^{40}K.

This validates fidelity to the training set, not ^{40}K background physics

The ^{40}K training set was recorded with a sphere-centre bug that kept only photons which had already Compton-backscattered off the structure above the detector before reaching the sphere — confirmed independently by the median crossing energy (211 keV, against the 217.5 keV backscatter energy of the 1461 keV line) and by the near-total absence of the unscattered line itself. Both the reference and MAGI draw on this same biased, back-scatter population, so the comparison below is a valid *closure* test of MAGI against what it was trained on, and **not** a statement about ^{40}K background flux. Ra-226 and Th-232, trained the same way, carry the identical caveat. A corrected retrain is planned; until it lands, DM1.2 and the CR source above carry every physics claim.

$R = 1.118 \pm 0.373$, Kolmogorov–Smirnov $p = 0.60$: 2×10^8 crossings injected on both sides (19 MAGI counts, 17 real) into the same mass model. ^{40}K closes at the detector at matched statistics. Separately, and independent of the training-population caveat above, MAGI turned the 1,873,609 real recorded crossings that set could ever supply into 538,205,667 generated ones — a 287× amplification of an otherwise statistically exhausted sample — and the amplified population still closes at the detector at the ratio above.

11.4.2 Memorisation test: does MAGI generate, or recall?

A large amplification factor does not by itself distinguish a generative model from a kernel-density resampler, which turns the same crossings into an unlimited sample by construction without learning anything. `memorisation_test.py` (§8.2) is the check: for each generated sample, the nearest-neighbour distance to a held-out reference set of real crossings, in the model's own feature space. A model that memorises places generated samples closer to training points than real data lies from itself; a model that generalises places them at the same distance real data sits from itself. The metric is calibrated in both directions before it is trusted — a held-out real split must read as real, and a battery of synthetic memorisers must read as recall — and the script refuses to report a verdict if either control fails.

	CR (ingoing-fixed)		DM1.2	
Arm	ratio	near-dup	ratio	near-dup
real → real (LOO)	1.000	1.00 %	1.000	1.00 %
held-out real (control)	1.001	0.90 %	0.999	1.07 %
MAGI	**1.027**	**0.89 %**	**1.536**	**0.34 %**
KDE, Silverman bandwidth	2.028	0.00 %	4.123	0.00 %
memoriser, jitter 0 (control)	0.000	100 %	0.000	100 %

MAGI does not memorise, on either source. Both ratios exceed 1, and both near-duplicate fractions sit *below* real data's own rate of coinciding with itself. CR lands almost exactly at the real-data value (1.027); DM1.2 shows the opposite failure instead, sitting ∼54 % too far from the training manifold, consistent with its larger marginal defects above — generation off the manifold, not memorisation of it. A kernel-density resampler (KDSource's approach) sits 2–5× further off the manifold than MAGI on the same metric in both sources tested, which is the trade behind its own correlation performance and its rate of physically-impossible samples.

11.4.3 Crossing-level: validated

All numbers below are produced by `tools/acceptance_v0_8.py` on the cosmic-ray source.

Quantity	**CR** (3 seeds)	**Bar**
Coupling, max $\lvert\Delta\text{corr}\rvert$	0.0285 ± 0.0101	≤ 0.05
Wasserstein on $\log_{10} E$	0.0241 ± 0.0046	≤ 0.05

The joint distribution holds up with error bars: every real cross-correlation over $(\log E, u_r, u_v, \phi_r, \phi_v)$ is reproduced inside the bar, on the cosmic-ray source, across seeds. The v0.7.2 categorical head does not achieve this — its coupling residual is 0.226, eight times worse.

11.4.4 Crossing-level: not validated

Per-line *intensities* range from in-band to a factor of 2× too high. Every figure below is traceable to `docs/_data/` output of the acceptance harness and belongs to v0.8.2; the 511 keV line of CR is 3.40 % of the generated γ against 3.45 % real in v1.0.0 (Section 7.1).

Line	**Recovery**	**Status**
CR Al Kα_1	0.959 ± 0.025	in band
CR Cu Kβ	1.034 ± 0.050	in band
CR e^+e^- 511 keV	1.825 ± 0.324	over-generated, unstable
CR Cu Kα_1	$2.052 \pm 0.034^\dagger$	over-generated

† **A routing fraction, not a window recovery.** Cu Kα_1 and Cu Kα_2 are 21 eV apart, so each sits inside the other's continuum side-bands and the harness reports the window-based `recovery_ratio` as `n/a` (`sideband_contaminated`). The number quoted is the harness's `comp_rec`: the fraction of generated events the *gate* routed to that component, against the real fraction. It measures the same failure and is the only figure available for this line, but it is not the same quantity as the other rows and should not be compared against the $[0.8, 1.2]$ window bar directly.

Practical consequences:

- Line *positions* are reliable — pinned physical inputs, audited to ≤ 0.5 eV against the measured spectrum. It is the number of events in each line that is wrong.
- Do not use generated output to estimate a fluorescence or annihilation line flux, or any quantity dominated by one.
- The continuum between lines, and the total energy marginal, are within the bar.
- Rare lines fare worse than strong ones, and the failure is source-dependent. Measure on *your* source with the acceptance harness rather than assuming these numbers transfer.

11.4.5 Choosing between the heads

Neither head is correct for lines today. v0.7.2 has a better energy marginal on the cosmic-ray source (0.0141 versus 0.0241) but produces **zero** events in the 511 keV window against 52 225 real, and its coupling is eight times worse. If your use depends on correlations, use v0.8.2. If it depends only on the marginal spectrum and you want the most conservative option, v0.7.2 remains available.

11.4.6 Closed: energy–geometry coupling does not explain the conversion deficit

The heads are conditionally independent given z, and the classifier two-sample test of §11.4.1 is sensitive to exactly this kind of joint mismatch. `energy_condition_geometry` conditions the energy head directly on the sampled geometry instead of only on z, which is the mechanism this defect calls for; see the note in Section 4. It shipped as the untested v0.8.3 feature and was not adopted on marginal grounds alone, which was too weak a verdict to trust on its own — those runs also changed `latent_dim` and used a contaminated training set, so the flag was never isolated.

A controlled A/B — identical data, split, transformers and seed, differing only in the flag — was scored two ways: by classifier AUC against 400,000 real crossings, and by injecting both checkpoints through the same detector chain.

Result: null, on both metrics. $\Delta\text{AUC} = -0.0062$, inside the pre-registered no-effect threshold. At the detector, $Q = R_{\text{coupled}}/R_{\text{baseline}} = 1.072 \pm 0.114$ in the X-IFU band — consistent with no change. The conditional-independence edge between the energy and geometry heads is not what drives the conversion deficit. `energy_condition_geometry` remains available (off by default) but is not recommended. The deficit itself is still open — characterized as a diffuse, joint, coverage-flavoured mismatch (§11.4.1), not yet mechanistically explained.

11.4.7 Negative results already established

Recorded so they are not re-attempted without new information:

- **Per-line gate class weights**, calibrated from measured routing: no effect distinguishable from noise. The untouched control line moved more (+27 %) than either weighted line.
- **Continuum polish** (CDF-warp knot boost, more flow bins): a three-seed grid put every difference under 1.2σ of the noise.
- **Exact-match gate labels** (`bandwidth_mode="exact"`): physically the more faithful label, but it did not fix the line it was built for and broke a confirmed result and the coupling bar. See the warning in Section 4 and `docs/v0.8.1_line_truth.md` §14.2.

12 Limitations and traps

The crossing surface is hard-limited to a sphere

`build_physical_features`'s `center`/`radius` and the transforms in `core/geometry.py` all assume that the surface on which the events are scored is a sphere. Adapting MAGI to a planar, cylindrical or box surface requires reworking the geometry-transform layer in both `preprocessing.py` and `generation/reconstruction.py`, which must stay in sync, and different parameters are not sufficient (Section 12.3).
This constrains the *scoring* surface only. The surface is an intermediate bookkeeping step in a multi-stage run; the practical workaround for an awkward geometry is usually to place a dummy spherical tracking volume around the region of interest to give MAGI a surface to learn on, and to leave the mass model itself alone.

12.1 Measured limits of v1.0.0

The numbers of this subsection are the measured state of v1.0.0, and the acceptance harness of the v0.8.2 beta has been superseded by Section 11. Each limit is stated with its measured size. A cause is given where it has been established, and the limits whose cause is open are marked as such.

Line positions are reliable and line intensities are source-dependent. The positions are pinned physical inputs audited to ≤ 0.5 eV against the measured spectrum on all sources (`tools/line_centroid_audit.py`). The intensity of a line is learned and is the least stable quantity: the beta measured recoveries from 0.7× to 4.9× the real value, unstable across seeds, and they have not been remeasured. The 511 keV line of CR, after the warp fix, is 3.40 % of the generated γ within 50 eV against 3.45 % real, and all 26 lines of Ra are modelled. A generated line flux is not to be used as an estimate of a fluorescence or annihilation line without measuring it on the source with `tools/acceptance_v0_8.py`, and a line below 5σ significance is correctly left unmodelled, and no recovery is quoted for it.

The prior does not reproduce the aggregated posterior closely enough. Approximately 80 % of the excess AUC of the original model came from this source, and the refit of Section 7.4 removes most of it. The remaining AUC of 0.62–0.63 on CR is due to the prior together with the per-variable heads.

Exact discrete atoms need a dedicated component. An atom that is not listed is spread by the continuum density over the neighbouring energies. The unscattered primary at $E = E_{\mathrm{prim}}$ on DM1.2 (53 % of the γ crossings) came out at 2.4 % before it received its own component, and with the atom and the zone table by primary it is 53.2–53.7 % (Section 5.1).

Tail defects of the SRON Th and Ra models. The Th warp2 model generates 722 crossings per 10^6 at 2.7 MeV against 21 real (0.07 %), the Ra tail between 2.6 and 4.0 MeV is 3.7 times too high, and the ratio between 20 and 40 keV is 0.43 for Th. The closures of the warp models give $R = 1.40 \pm 0.22$ for Th and 1.31 ± 0.19 for Ra at SRON, while the IAPS models give 0.88 and 0.86 (Section 11.3), and both are declared.

Grazing entries. About 5 % of the u_r of Th and Ra points outward, and the grazing entries on DM1.2 (cosine with the inward normal below 0.1) are approximately three times too many. This is not an effect of the width of the head, which was scanned, and it is declared. The Th232 ϕ_v deviation of 5–6 in every arm has an open cause, either the head or the transform, and no experiment has separated them.

Crossings are independent given the conditioning. The correlations inside one shower are not represented: a single shower of 1364 crossings, of which 492 pass through a spot of approximately 1 mm with a common direction, appears in the CR file and MAGI does not reproduce it, and on DM1.2 the energies of the k crossings of a primary can sum to more than E_{prim}. The energy loss flow keeps the own crossing of the primary below E_{prim} by construction, but nothing in the model enforces the budget of the secondaries. The autoregressive follow-up removes this limit.

The energy head misplaces the muons between 10 and 100 MeV for low E_{prim} on DM1.2 (too few at 10–30 MeV, too many at 60–100 MeV), in the posterior arm as well; neither the refit nor the bank removes it, and the cause is open (the head or the transform of the energy). The condition-matched AUC is 0.68 for the γ primaries and 0.63 for the μ^- primaries, against a null of 0.50.

Open items of the 1.0.1 patch. The Th232 and μ `_qtfull` closures at 5x, the quantile swap of the SRON models (which needs the split dumps), the K-40 delta and the seeds 8 and 9 of the validation. The sources in passive material and the Athena mass model (Section 6) are not covered.

12.2 Traps

The training data is a crossing spectrum and not a deposited energy. Events are recorded where they cross a virtual sphere, with no detector response applied. Classical detector escape peaks may legitimately be absent, and a null result there is a real result.

`geometry_transform` mismatches do not raise. They produce physically wrong output. The string is persisted in the metadata for this reason, and the metadata should be passed on instead of the string being re-specified by hand.

Particle-type filtering silently drops rare species. If a rare but relevant type is missing from the generated output, `prob_threshold` is the first place to check.

Energy binning limits which lines can be *found*. With the mixture head the binning no longer limits where a line can be *placed*, but on a log grid spanning many decades a bin

can be hundreds of eV wide, and close doublets fall in one bin. Use `refine_bin_width_mev` and `confirm_unresolved_candidate_lines`, then verify with `tools/line_centroid_audit.py`.

Quantile transforms need enough samples. Small datasets give unreliable tail behaviour in the fitted transform, which shows up as implausible extreme values in the generated output (Section 7.3).

Flux normalization depends on `source_type`. A wrong choice does not raise an error. Radioactive sources additionally need the 13-column lineage schema.

Unit tests cover the machinery and not the physics. The 238 passing tests of `MAGI_package/tests/` do not tell whether a trained model reproduced the source, which is the purpose of Section 10.

CPU-only by default. On this workload `tensorflow-metal` is slower than the CPU.

12.3 Adapting MAGI to another detector or source

The `center` and `radius` parameters of `build_physical_features` and the coordinate transforms of `core/geometry.py` (`xy_from_ur_phi`, `vxyz_from_uv_phi` and others, all parametrised by `sphere_R`) assume a sphere centred at a fixed point. A planar, cylindrical or box-shaped surface requires reworking the geometry-transform layer, both the physical features of `preprocessing.py` and the reconstruction of `generation/reconstruction.py`, which have to stay in sync. The settings below need attention on any new source.

`geometry_transform` is a contract between training and generation. The string (`"arctanh_uv_discrete"`, `"quantile_u_r_u_v"`, `"quantile_u_r_u_v_phi_r_phi_v"`, `"quantile_u_r_mu_phi_r_psi"`) and the resulting ordering of `X_cont_cols` are shared by the preprocessing, the input layout of the model and the reconstruction, and they are stored in the metadata. They have to match exactly between the run that trained a model and the run that generates from it, since no check catches a mismatch early and the result is a physically wrong output.

The energy-binning mode affects the resolution of the lines. `build_energy_bins` supports `"fixed_width"`, `"min_counts"` and `"log_fixed_count"`, and the choice trades the ability of a categorical head to resolve narrow lines against the statistics of every bin. `refine_bin_width_mev` refines each detected peak to sub-bin precision, and `tools/line_centroid_audit.py` checks the result.

The flux normalization depends on `source_type`. `source_type="cosmic_ray"` (default) takes a crossing as primary when `PrimBool == 1`, that is when the raw particle thrown by the GPS reached the surface unmodified, and `source_type="radioactive"` takes it as primary when `CreatorProcessName == "RadioactiveDecay"`. The second mode is needed due to the literal primary of a decaying source embedded in bulk material being the decaying nucleus, which never propagates; `PrimBool` is therefore always 0. `compute_primary_fraction`, `save_normalization_summary` and `load_normalization_summary` measure and store the factor.

Any change of the geometry-transform layer has to be validated by hand against real data before the generated output is trusted, since the unit tests cover the machinery only.

12.4 Resuming an interrupted or early-stopped run

A fit can end before it is finished, due to a killed process, an expired Colab session, or `EarlyStopping` fires a few epochs too soon. Passing `checkpoint_dir` to `fit_model` (or `train_single_run`) writes a resumable checkpoint after every epoch, and `resume=True` continues from it.

```
history = magi.fit_model(
    model, train_ds, val_ds, epochs=40,
    callbacks=magi.build_default_callbacks(),
    checkpoint_dir="trained_models/my_run/resume",
    resume=True,
)
```

Running the same call again after an interruption picks up at the last completed epoch, and when `checkpoint_dir` holds nothing yet, training starts from scratch; one command serves for the first run and for every restart. `epochs` is the total, as in Keras, and the returned history covers all epochs, including those before the interruption. The model has to be compiled with the same optimizer and `clipnorm` as the interrupted run, and a mismatch raises an error.

- **Restored state.** The weights, the Adam moments and step count, the learning rate, the state of `EarlyStopping` (best value, patience counter, best weights), `ReduceLROnPlateau` and `TaskAdaptiveLossScheduler`, the decayed `model.task_weights` and the metric history. With an unshuffled dataset a resumed run reproduces an uninterrupted one weight for weight (`tests/test_resume_training.py`), while with shuffled data it is equivalent statistically, since the order of the batches is not restored.
- **Early stopping ended too soon.** Raise `epochs`, optionally raise `patience` of the `EarlyStopping` instance, and pass `reset_early_stopping=True`. The patience counter is cleared and the best value found so far is kept, and the run continues from the best weights.
- **Kill during the write.** The checkpoint is written to a temporary directory and swapped in; an interruption leaves the checkpoint of the previous epoch intact. `magi.load_resume_state(dir)` reads how far a run got without loading a model.

The resume checkpoint is a training aid and is not a model artifact: generation still needs the files of `save_final_trained_model`, which are written when the fit ends. Jobs longer than the limit of a tool or session are run detached (`python tools/run_detached.py <log> <cmd...>`) together with the per-epoch checkpoint.

13 Where to look next

Document	Contents
`MAGI_package/docs/USAGE.md`	Short-form version of this manual, the reference for v1.0.0.
`CHANGELOG.md`	What changed in 1.0.0 and the known limits left for 1.0.1.
`CITATION.cff`	The citation metadata of the release.
`Example_Usage.ipynb`	Runnable walkthrough on synthetic data.
`docs/CVAE_vs_AutoRegFlow.md`	The comparison with the autoregressive follow-up, including the global frame against the local direction.
`docs/TrackG_design_review.md`	The design of the first-entry crossings (Section 6).
`docs/v1.0.0_error_budget.md` and `docs/v1_0_rms_validation.md`	The error budget and the per-bin RMS validation behind Section 11.
`docs/v0.8.1_line_truth.md`, `docs/v0.8.2_RoadmapForAdoption.md`, `docs/v0.8_learnable_prior_theory.md`, `docs/v0.8_v072_comparison.md`	History of the v0.8 line: the measurement record of the beta, its adoption plan, the reason for the conditional prior and the head-to-head comparison, whose absolute numbers predate the metric fix.
`docs/MAGI_state_reference.tex`	The full, provenance-tagged record behind the DM1.2/SRON/memorisation numbers in Section 11.4.1: every superseded and withdrawn result, and what may and may not be quoted from each.
`docs/_data/memorisation_cr.txt`, `docs/_data/memorisation_dm12.txt`	Raw `memorisation_test.py` output behind §11.4.2.

Reporting a problem

Include the run directory's `*_config.json` and `*_metadata.json`, the acceptance-harness JSON if you have it, and the seed. Most surprising results trace back to a preprocessing contract mismatch that those files make visible immediately.

14 Citing MAGI, and its license

14.1 License

MAGI is released under the **GNU General Public License v3.0** (GPL-3.0-only) — the same license used by KDSource, the closest published analogue to this tool. The full text is in `LICENSE` at the repository root. In short: you may use, modify and redistribute MAGI freely, including commercially, but any distributed derivative work must also be licensed under the GPL with its source made available, and MAGI is provided with no warranty.

14.2 Citation

If MAGI contributed to a publication, please cite it. The canonical, up-to-date citation metadata lives in `CITATION.cff` at the repository root — readable directly by GitHub's "Cite this repository" button and by tools such as `cffconvert` — and takes precedence over the text below if the two ever disagree. At the time of writing it resolves to:

```
Monastra, F. (2026). MAGI: Multivariate Autoencoder for particle
```

```
Generative Inference (Version 1.0.0) [Computer software].
https://github.com/francescomonastra/MAGI
```

No archival DOI yet

MAGI does not yet have a version-of-record deposited with an archiving service such as Zenodo. Cite the repository and version above until one exists; `CITATION.cff` carries the release date of the tag, and a `preferred-citation` entry is added once a paper describing MAGI is published.

References

[1] Diederik P. Kingma and Max Welling. Auto-Encoding Variational Bayes. In *International Conference on Learning Representations (ICLR)*, 2014.

[2] Kihyuk Sohn, Honglak Lee, and Xinchen Yan. Learning Structured Output Representation using Deep Conditional Generative Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, 2015.

[3] S. Agostinelli et al. Geant4—a simulation toolkit. *Nuclear Instruments and Methods in Physics Research A*, 506(3):250–303, 2003.

[4] J. Allison et al. Recent developments in Geant4. *Nuclear Instruments and Methods in Physics Research A*, 835:186–225, 2016.

[5] George Papamakarios, Eric Nalisnick, Danilo J. Rezende, Shakir Mohamed, and Balaji Lakshminarayanan. Normalizing Flows for Probabilistic Modeling and Inference. *Journal of Machine Learning Research*, 22(57):1–64, 2021.

[6] Ivan Kobyzev, Simon J.D. Prince, and Marcus A. Brubaker. Normalizing Flows: An Introduction and Review of Current Methods. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(11):3964–3979, 2021.

[7] Conor Durkan, Artur Bekasov, Iain Murray, and George Papamakarios. Neural Spline Flows. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019. Monotonic rational-quadratic spline transforms; the continuum density in magi/core/flows.py.

[8] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal Loss for Dense Object Detection. In *IEEE International Conference on Computer Vision (ICCV)*, 2017. Focal down-weighting of easy examples; the gate loss must route lines that are 1e-5 of the sample.

[9] Jakub M. Tomczak and Max Welling. VAE with a VampPrior. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2018. The aggregated-posterior/prior mismatch argument for replacing N(0,I) with a learned prior.

[10] Laurent Dinh, Jascha Sohl-Dickstein, and Samy Bengio. Density Estimation using Real NVP. In *International Conference on Learning Representations (ICLR)*, 2017. Affine coupling layers; the architectural basis of the conditional prior in magi/core/priors.py.

[11] J. A. Bearden. X-Ray Wavelengths. *Reviews of Modern Physics*, 39(1):78–124, 1967. Source of the transition labels in the candidate-line table.

[12] S. T. Perkins, D. E. Cullen, M. H. Chen, J. H. Hubbell, J. Rathkopf, and J. Scofield. Tables and Graphs of Atomic Subshell and Relaxation Data Derived from the LLNL Evaluated Atomic Data Library (EADL), Z = 1–100. Technical Report UCRL-50400, Vol. 30, Lawrence Livermore National Laboratory, 1991. The fluorescence energies Geant4 actually emits, and therefore the ones MAGI pins its line components at.

[13] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations (ICLR)*, 2015. Default optimizer in magi.compile_model.

[14] N. S. Schmidt, O. I. Abbate, Z. M. Prieto, J. I. Robledo, J. I. Márquez Damián, A. A. Márquez, and J. Dawidowski. KDSource, a tool for the generation of Monte Carlo particle sources using kernel density estimation. *Annals of Nuclear Energy*, 177:109309, 2022.

[15] Baran Hashemi and Claudius Krause. Deep generative models for detector signature simulation: A taxonomic review. *Reviews in Physics*, 12:100092, 2024.

[16] Michela Paganini, Luke de Oliveira, and Benjamin Nachman. CaloGAN: Simulating 3D high energy particle showers in multilayer electromagnetic calorimeters with generative adversarial networks. *Physical Review D*, 97(1):014021, 2018.

[17] Claudius Krause and David Shih. Fast and accurate simulations of calorimeter showers with normalizing flows. *Physical Review D*, 107(11):113003, 2023.

[18] ATLAS Collaboration. Deep generative models for fast photon shower simulation in ATLAS. *Computing and Software for Big Science*, 8:7, 2024.

[19] S. Lotti et al. Review of the Particle Background of the Athena X-IFU Instrument. *The Astrophysical Journal*, 909(2):111, 2021.

[20] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic Attribution for Deep Networks. In *International Conference on Machine Learning (ICML)*, volume 70, pages 3319–3328, 2017. Integrated Gradients; the input-feature attribution that conductance generalizes to internal units.

[21] Kedar Dhamdhere, Mukund Sundararajan, and Qiqi Yan. How Important Is a Neuron? In *International Conference on Learning Representations (ICLR)*, 2019. Conductance. The usage metric behind magi.save_full_circuit.

[22] Gabriel Erion, Joseph D. Janizek, Pascal Sturmfels, Scott M. Lundberg, and Su-In Lee. Improving performance of deep learning models with axiomatic attribution priors and expected gradients. *Nature Machine Intelligence*, 3:620–631, 2021. Expected gradients: baselines drawn from the data distribution instead of a single arbitrary reference.